

Confronto Parallela vs Seriale: Lo Strobe e lo Start Bit

- **Parallela:** Necessita di un segnale di **strobe**. Senza di esso, se vengono messi due dati uguali in successione, il sistema non saprebbe distinguerli.
 - **Seriale (RS-232):** Non ha bisogno dello strobe per distinguere byte successivi.
 - In riposo la linea è **alta (1)**.
 - **Il bit di Start è 0.**
 - Se arrivano due byte uguali, ci si accorge della distinzione perché si vedono due bit di start.
-

Gestione degli Errori e Sincronizzazione

Mentre **nella parallela non è previsto dal protocollo la gestione degli errori** (che va risolta a livello algoritmico), **nella seriale è fondamentale gestire tre aspetti:** **Aspetto fisico** (perché accade), **Detection** (come capirlo) ed **Evitare l'errore**.

A. Rumore e Bit di Parità

- **Il rumore è ineliminabile in ogni sistema di trasmissione**, si può solo ridurre la probabilità con schermature o rendendo i sistemi equipotenziali.
- **Detection:** Si usa il **bit di parità**.
- **Limite:** Se si corrompono due bit contemporaneamente, il sistema di parità non funziona.

B. Sincronizzazione di Fase e Frequenza

- **È impossibile avere due sistemi con clock perfettamente identici (isocroni)**; quando la frequenza aumenta, lo sfasamento tra le linee aumenta.
- Ognuno ha il suo clock, **quindi lo sfasamento è un errore non eliminabile**.

C. Overrunning

- **È l'unico errore eliminabile.**
- **Nella parallela non esiste grazie allo strobe**; nella seriale, mancando lo strobe, il dato può essere sovrascritto.
- **Soluzione:** Si può ovviare con l'**handshaking** facendolo per ogni byte trasmesso. Invece se lo facciamo una volta sola all'inizio possiamo avere questo problema.

D. Errore di Framing (Framing Error):

- Si verifica quando il ricevitore non "vede" il bit di Stop nel momento esatto in cui se lo aspetta alla fine di un pacchetto di dati. Questo segnala tipicamente una perdita di sincronizzazione tra trasmettitore e ricevitore, spesso causata da baud rate configurati in modo diverso sui due dispositivi o da forti disturbi sulla linea che alterano i tempi di lettura.
-

L'Handshaking e la RS-232

L'handshaking garantisce che non si perda il dato, **ma se fatto per ogni singolo carattere rende il sistema poco efficiente.**

- **Quando si fa:** Solitamente solo all'inizio della comunicazione di un messaggio.
- **Segnali:** Sono **4 segnali** (2 che vanno e 2 che tornano).
- **Origine:** Il sistema nasceva per i modem: il primo impulso serviva per verificare se la linea fosse attiva, il secondo per mandare il messaggio.

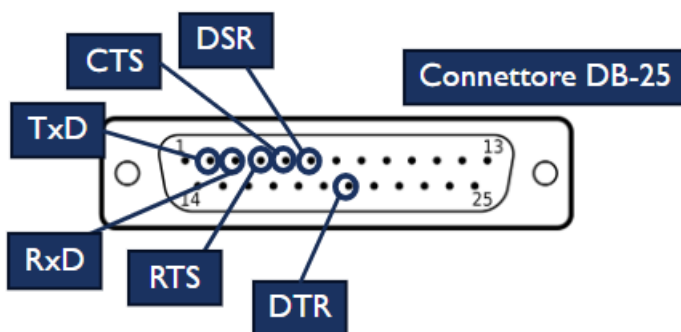
Interfaccia al BUS e Componente Intel 8251A

L'interfaccia seriale verso il BUS si compone di diverse parti (Modo, Controllo, Dati).

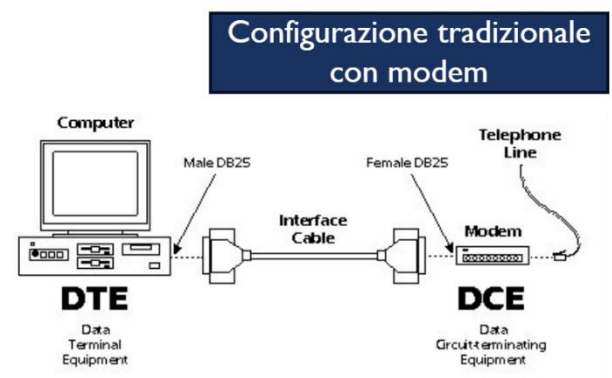
Gestione sotto Interruzione

L'interruzione può avvenire sia durante l'handshaking, sia ad ogni carattere letto.

- Ogni volta che arriva un'interruzione, il sistema sa che deve leggere il dato.
- **Rischio Overrun:** Se il dato non viene letto in tempo, l'altro continua a mandare e sovrascrive.
- **Buffer:** Abbiamo comunque un po' di tempo per intervenire perché sono presenti **due buffer**.
- **Connessione:** Il sistema termina su un connettore standard.



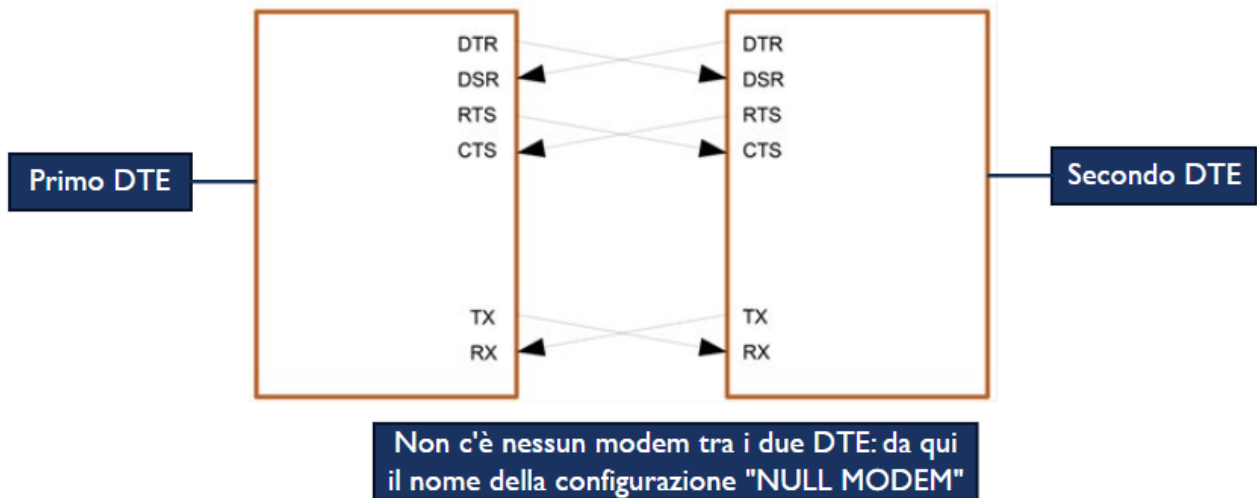
- Questo standard prevede l'interfacciamento tra un Data Terminal Equipment (DTE) e un Data Communication Equipment (DCE)
 - Un DTE potrebbe essere un computer
 - Un DCE potrebbe essere un modem
- L'interfacciamento avviene tramite un set di segnali di dato e di controllo
 - Data Terminal Ready (DTR)
 - Data Set Ready (DSR)
 - Request To Send (RTS)
 - Clear To Send (CTS)
 - Transmitted Data (TxD)
 - Received Data (RxD)
- I segnali DTR, DSR, RTS e CTS servono a realizzare l'handshaking tra DTE e DCE



Lo standard RS-232 non supporta nativamente una configurazione in cui due DTE siano direttamente collegati

Per ovviare a questa mancanza, si ricorre a una configurazione ad-hoc: **la configurazione NULL MODEM**

Dati due DTE, i segnali DTR, DSR, RTS, CTS, TxD e RxD **sono intrecciati**



I Due segnali che vanno sono DTR e RTS che sono Data Terminal Ready (terminale pronto) e Request to send (richiesta di invio). I due segnali che entrano invece sono DSR (data set ready) e CTS (clear to send).

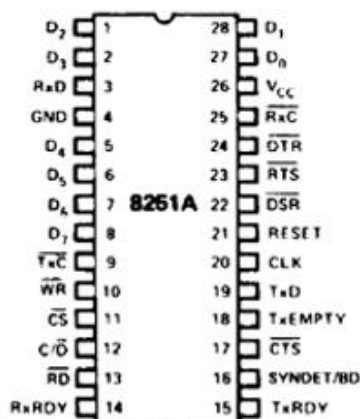
Possiamo dire che c'è un doppio handshaking che come detto prima è solo dovuto al fatto che si doveva prima vedere se sulla linea c'era qualcuno.

Collegiamo due oggetti che sono due computer.

All'intero del BUS si collega come la parallela. All'esterno invece cambia.

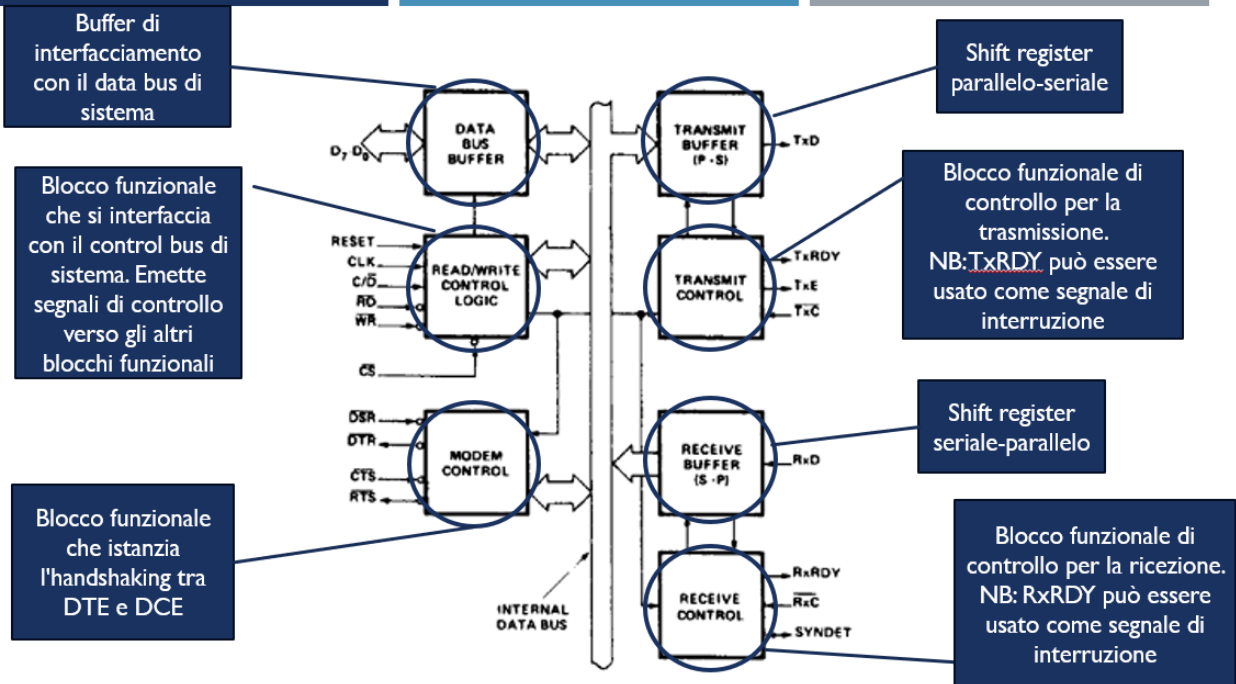
Il componente Intel 8251A è la prima periferica USART apparsa sul mercato abilitata a interfacciarsi con i microprocessori Intel

Questo USART può gestire le interazioni **secondo lo standard RS-232**



Se devo collegare due oggetti devono avere un'interfaccia ognuno, avrei bisogno di comprare questo componente per ogni oggetto. È una follia. Allora esistono altri protocolli.

Architettura dell'Intel 8251A



In totale sono 28 PIN

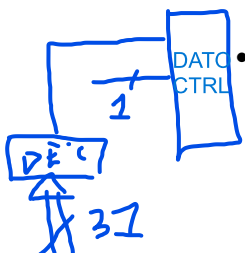
- 1) **Il primo blocco sono i pin che vanno da D0 a D7** cioè 8 bit che sono quelli che si interfacciano con il **DATA BUS** interno. Sulla seriale vediamo che questi pin sono un po' dislocati, mentre nella parallela erano tutti vicini, ma comunque possiamo considerarli come un blocco di 8 bit. **La CPU deposita il dato sul Bus Dati. Il blocco Data Bus Buffer lo accetta e lo mette sul bus interno del chip.** In questo istante, il dato è ancora in formato parallelo (tutti gli 8 bit sono presenti contemporaneamente). Quindi lato CPU viene trasmesso in parallelo, come per la parallelo. Sarà poi dopo che viene fatto uscire in modo differenti cioè secondo un protocollo seriale. Quindi mancano da comprendere altri 20 bit.
- 2) C'è un altro blocco che si interfaccia con il BUS di sistema e nello specifico con il **CONTROL BUS**: è il **Read/write control logic**. **È lui che decide se il chip deve parlare con il BUS di sistema, in che direzione deve andare il dato e se quel dato è un'istruzione o un'informazione pura.** D, WR, RD lavorano sul fronte di discesa: è questo il motivo per il quali sono barrati sopra.

Questo blocco riceve i segnali dal BUS di controllo della CPU e li traduce in azioni interne:

- **CS* (Chip Select):** È la porta d'ingresso. **Il Chip Select è fondamentale perché, in un computer, la CPU non è collegata solo alla USART, ma a tantissime altre periferiche** (RAM, ROM, Timer, altre porte) **tutte sullo stesso BUS dati.** **CS* = 0 (Basso):** Il chip è "sveglio". È selezionato per l'operazione e reagirà ai segnali di lettura o scrittura. Senza il Chip Select, se la CPU mandasse un ordine di scrittura, tutte le periferiche proverebbero a leggere contemporaneamente, creando il caos. Se questo segnale non è a 0, il chip è "tagliato fuori" dal bus. Serve a permettere alla CPU di gestire più periferiche sullo stesso bus: solo quella con il CS attivo risponderà.

• **C/D* (Control/Data):** Questo pin è il selettore di destinazione interna. Corrisponde al bit meno significativo dell'indirizzo e quindi indirizzo pari o dispari.

- Quando la CPU manda un dato, **se questo pin è 1**, il dato va nel registro di **Controllo** (configurazione del Modo o Comando).
- **Se questo pin è 0**, il dato va nel buffer per essere **Trasmesso** (o letto dalla Ricezione).



- **Read:** Quando va a 0 la CPU informa il chip che sta leggendo i dati o lo stato
- **Write:** Quando va a 0, la cpu informa il chip che sta scrivendo il dato o lo stato

RD e WR sono entrambi segnali generati dalla CPU.

(Write): Dalla CPU Alla USART

- **Significato:** La CPU dice: "Ehi USART, **ti sto scrivendo** qualcosa".
- **Azione:** La CPU mette il dato sul bus e abbassa WR. La USART "sente" il fronte di discesa e cattura i bit dai pin D0-D7 per portarli al suo interno (verso lo Shift Register o i registri di comando).

(Read): Dalla Alla USART

- **Significato:** La CPU dice: "Ehi USART, **voglio leggere** cosa hai dentro".
- **Azione:** La CPU abbassa RD. Solo a quel punto la USART "apre i rubinetti" (i buffer tri-state) e mette il contenuto del suo registro di ricezione o di stato sul bus dati. La CPU poi "campiona" quel dato dal bus.

C/D	RD	WR	CS	
0	0	1	0	8251A DATA → DATA BUS
0	1	0	0	DATA BUS → 8251A DATA
1	0	1	0	STATUS → DATA BUS
1	1	0	0	DATA BUS → CONTROL
X	1	1	0	DATA BUS → 3-STATE
X	X	X	1	DATA BUS → 3-STATE

Esempio: C/D* = 0 quindi Dato, RD*=0 e WR*=1 quindi la cpu informa il chip che sta leggendo. Quindi alla fine la cpu sta leggendo il dato che si trova nel buffer dato del chip. Tutto questo vale se CP*= cioè questa periferica è attivata.

- **RESET:** Un segnale alto qui riporta il chip nello stato "Idle", pronto per una nuova configurazione (il "Modo").
- **CLK:** È il clock interno che sincronizza la logica di questo blocco con il resto del sistema.

Abbiamo visto quindi 8 pin del dato + 6 pin di controllo CPU → periferica = 14 pin. Mancano altri 14 pin.

3) **Modem Control (Handshaking RS-232): sono i 4 segnali per parlare con un modem o gestire il flusso. Gestiscono l'handshacking tra DTE e DCE. Sono tutti attivi bassi (hanno la barretta sopra):**

- **DSR* / DTR*:** (Data Set Ready / Data Terminal Ready). Data terminal ready esce dalla UART e serve a dire "Il terminal è pronto". Arriva nel DCE che risponde sulla linea DSR per dire "Anche io sono acceso".
- **RTS* / CTS*:** (Request to Send / Clear to Send) Servono per chiedere il permesso di trasmettere e riceverlo. RTS parte dal DTE per dire "Ho dei dati pronti nel buffer, posso mandarteli?". Il dispositivo ricevente risponde: "Sì, la via è libera, spara pure i dati".

Siamo arrivati a 14 + 4= 18 pin. Mancano altri 10 pin da studiare.

Ci manca la parte relativa alla trasmissione e la ricezione. Oltre ai pin su cui escono effettivamente i singoli bit seriali, oppure vengono ricevuti, ci sono dei blocchi che controllano la ricezione e la trasmissione

- 4) **Buffer di trasmissione:** Pin Tx su questo bit vengono fatti uscire i bit in modo seriale uno alla volta. È uno shift register parallelo seriale. Significa che ha 8 registri, in cui possiamo mettere gli 8 bit dentro. E poi l'uscita è solo l'ultimo bit. Poi shiftiamo e c'è in uscita un altro bit.
- 5) **Blocco del controllo di trasmissione:**

- **TxC (Transmitter Clock):** Questo è il clock esterno che entra nella USART. Determina la velocità con cui i bit vengono "buttati fuori" sul pin **TxD**. Come notavi nei tuoi appunti sulla sincronizzazione, trasmettitore e ricevitore devono avere la stessa frequenza (isocronismo). Nell'8251A, il baud rate può essere una frazione del clock (es. 1/16 o 1/64), parametro che si imposta nella fase di configurazione del **Modo**.
- Dialogo con la CPU: **TxRDY (Transmitter Ready)**. Questo segnale è fondamentale per la gestione a **interruzione** di cui parlavamo prima. Il buffer è considerato "vuoto" (e quindi **TxRDY = 1**) non appena il dato passa dal *Buffer di Trasmissione* allo *Shift Register*.

Cosa dice: "Il mio buffer interno è vuoto, la CPU può scrivermi un nuovo byte".

Utilità: Evita che la CPU sprechi tempo a controllare continuamente (polling). Quando il pin **TxRDY** diventa alto, scatta l'interruzione e la CPU sa che può caricare il dato successivo.

- Stato finale: **TxEMPTY (Transmitter Empty):** C'è una sottile differenza con **TxRDY**:

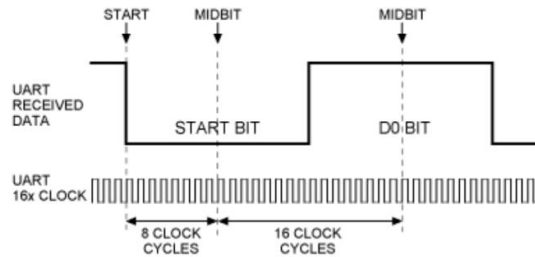
TxRDY si attiva appena il buffer di tenuta è libero (ma il chip potrebbe star ancora trasmettendo i bit dell'ultimo byte).

TxEMPTY si attiva solo quando **tutto** è stato inviato e non ci sono più bit in "viaggio" sulla linea. Quando ha finito di inviare tutto il byte, ha lo shift register vuoto, quindi si può caricare un altro byte dal buffer dato nel buffer di trasmissione. Non lancia un'interruzione, è una questione hardware interna del dispositivo seriale.

- 6) **Buffer di ricezione:** Pin Rx su questo bit vengono ricevuti i bit in modo seriale uno alla volta. È uno shift register seriale parallelo. Significa che ha 8 registri, in cui si riceve il bit e si shifta. L'uscita è presa in parallelo. Se notiamo la freccia è verso il BUS DATI del dispositivo infatti i bit vengono dall'esterno e devono essere presi.

7) **Blocco del controllo di ricezione**

- **RxC (Receiver Clock):** Per gestire il problema della fase e della frequenza (sistemi non isocroni), il ricevitore usa il clock **RxC**. Il chip non guarda il bit una sola volta. Spesso campiona il segnale a una frequenza molto più alta (es. 16x rispetto al baud rate). Quando vede il bit di Start (0), inizia a contare i cicli di clock per andare a leggere i bit successivi esattamente al centro della loro durata. Questo serve a "capire" il dato anche se i clock di chi trasmette e chi riceve sono leggermente sfasati.



- La velocità di trasmissione può essere configurata in base alla baud rate, che tipicamente è compresa fra 9600 e 115200; entrambi i dispositivi comunicanti **devono essere configurati con la stessa baud rate**
- Per mantenere la sincronizzazione, il **ricevitore campiona il segnale** in arrivo su RX **ad una frequenza superiore** (16x o 64x) rispetto a quella di invio
- Appena riconosciuto lo start bit, il ricevitore **si posiziona a centro bit** «contando» un certo numero di fronti di clock;
- I successivi campionamenti vengono effettuati posizionandosi sempre a centro bit per minimizzare gli errori

- **RxRDY (Receiver Ready): L'allarme per la CPU. È il segnale di interruzione più critico.**

Cosa dice: "Ho ricevuto un byte completo, l'ho messo nel buffer di ricezione. Vieni a prendertelo subito!".

L'output di questo pin indica che il chip contiene tutto il carattere che è pronto per essere preso dalla CPU. Questo PIN può essere collegato all'interrupt oppure essere letto in polling dalla CPU tramite un'operazione di lettura di stato.

Se fallisci nel leggere il carattere dal registro prima che il *prossimo* sia assemblato, scatta l'**overrun condition error**.

- **SYNDET / BD (Sync Detect / Break Detect)**

Questo pin ha una doppia funzione:

- In modalità sincrona serve per la sincronizzazione.
- In modalità asincrona (RS-232), può segnalare il **Break**: ovvero quando la linea rimane a 0 per un tempo molto lungo (più di un intero frame), segno che la connessione è caduta o interrotta fisicamente.

Quindi eravamo a 18 pin, ne abbiamo visti altri 8 siamo a 26. Ne mancano 2: VDD cioè alimentazione e GND cioè massa. È impossibile ricordarli a memoria, ma comunque basta ricordarsi che 8 pin sono quelli del dato collegato sul BUS interno della CPU, poi 4 segnali di handshaking, poi i segnali principali di controllo (Reset, clock sono banali, gli altri 4 cioè RD, WR, CS, C/D servono per le modalità di funzionamento) e poi ci sono i blocchi di trasmissione e ricezione.

Nell'ambito del seriale esistono diverse possibilità: RS-232 oppure I²C (Isquarec).

Come si setta il modo?

Per usare la periferica dobbiamo programmarla e quali sono i registri.

Il modello di programmazione della USART consta dei registri accessibili dal programmatore per configurare la periferica e comunicare/ricevere dati verso/dalla periferica.

La distinzione degli Indirizzi (Pin C/D*)

Come abbiamo visto, il pin (Control/Data) decide a quale "famiglia" di registri accedere:

- **Indirizzo Pari C/D*=0:** Si accede al **DATA register**. Qui scrivi i byte da spedire o leggi quelli ricevuti. È una linea diretta.
- **Indirizzo Dispari C/D*=1:** Qui le cose si complicano. **Questo indirizzo è condiviso da MODE, CONTROL, STATUS e SYNC.**

L'ordine di accesso (in scrittura) ai registri MODE, SYNC 1, SYNC 2 e CNTRL segue un ben preciso schema. Ricordando la PIA anche lì funzionava allo stesso modo. Se mettevamo 0 nel bit 2 del Registro di controllo allora l'altro registro era il registro direzione, altrimenti se mettiamo 1 era il registro dato.

Control Words (Parole di Controllo)

Prima di iniziare la trasmissione o ricezione dei dati, l'8251A deve essere caricato con un set di parole di controllo generate dalla CPU. Questi segnali definiscono la funzione completa dell'8251A e devono seguire immediatamente un'operazione di Reset (interno o esterno). Le parole di controllo sono divise in due formati:

1. **Mode Instruction** (Istruzione di Modo)
2. **Command Instruction** (Istruzione di Comando)

1) **Mode Instruction (Istruzione di Modo)**

Questa istruzione definisce le caratteristiche operative generali dell'8251A. Deve seguire un'operazione di Reset. Una volta che la Mode Instruction è stata scritta, possono essere scritte le istruzioni SYNC o le Command Instructions.

2) **Command Instruction (Istruzione di Comando)**

Questa istruzione definisce una parola che viene usata per controllare l'effettiva operazione dell'8251A. **Definisce cosa deve fare il chip adesso. Ad esempio: "Abilita il trasmettitore", "Abilita il ricevitore" o "Resetta i flag di errore".** Sia la Mode che la Command devono conformarsi a una sequenza specifica per il corretto funzionamento del dispositivo. La Mode Instruction deve essere scritta immediatamente dopo un Reset, prima di usare l'8251A per la comunicazione dati.

Per tornare al formato Mode Instruction, si può impostare il bit di **Master Reset** nella Command Instruction, il che riporta automaticamente l'8251A nel formato Mode Instruction.

Sequenza dei passi

Questa immagine è uno **schema temporale logico** che mostra l'ordine in cui la CPU deve inviare i dati al chip per farlo funzionare. Immaginala come una coda agli sportelli:

1. **In alto C/D*=1:** Noti che le prime istruzioni hanno tutte il pin a 1 (indirizzo dispari). La primissima è la **MODE INSTRUCTION**. Senza questa, il chip non sa nemmeno se deve lavorare in asincrono (RS-232) o sincrono.
2. **Parentesi SYNC:** Se nella Mode hai scelto la modalità sincrona, il chip si aspetta obbligatoriamente uno o due caratteri SYNC. Se hai scelto asincrono, questi vengono saltati.
3. **COMMAND INSTRUCTION C/D*=1 :** Una volta configurato il "come" (Mode), invii il comando per dire "inizia a lavorare" (Enable Tx/Rx).
4. **DATA (C/D*=0):** Solo adesso puoi passare all'indirizzo pari per inviare o ricevere i dati veri e propri.

5. **Alternanza:** Nota che mentre i **DATA** vanno sempre all'indirizzo pari, puoi inviare una nuova **COMMAND** all'indirizzo dispari in qualsiasi momento (ad esempio per fermare la trasmissione), ma per cambiare il **MODE** devi per forza resettare tutto (tornare in cima allo schema).

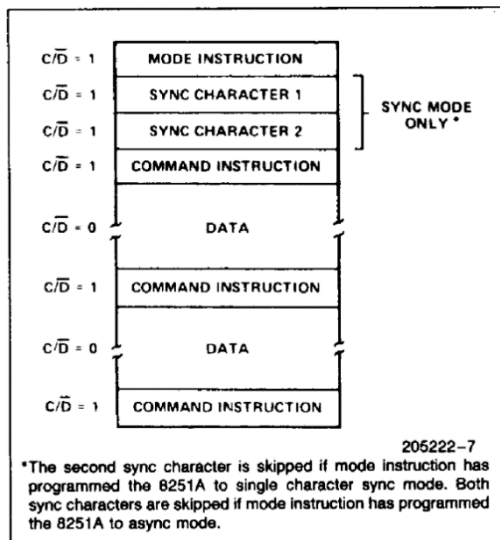


Figure 7. Typical Data Block



L'accesso all'indirizzo dispari è governato da questo schema

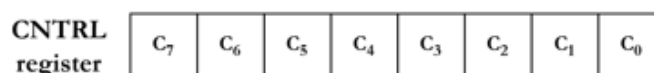
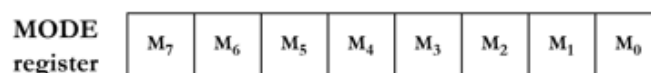
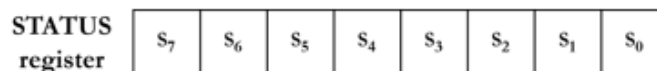
L'automa si interpreta in questo modo:

- 1) Stato di reset: il chip punta all'indirizzo MODE. Appena la CPU fa la prima write ($WR^*=0$) il dato viene scritto nel MODE e l'automa va avanti in base al fatto che nel mode ho selezionato la modalità sincrona o asincrona. (appena scrivo, va sempre avanti, non rimane nel MODE)
- 2) Una volta arrivati nel controllo, continueremo a rimanere lì se la CPU scrive. Se mettessimo $C/D^*=0$ andiamo nel canale parallelo e quell'automa rimane bloccato, quindi se poi torniamo all'indirizzo dispari ripartiamo dallo stesso stato.

Nota che se nel carattere SYNC1 mettiamo 1, cioè modalità sincrona singola, allora il secondo è skippato. Ovviamente se la modalità è asincrona sono skippati entrambi.

Lo Status Register condivide lo stesso indirizzo fisico del MODE e del CNTRL (quello dispari, es. **8001**). Il chip distingue tra loro usando il segnale RD(Read). Via software significa effettuare lettura o scrittura.

- Se scrivi ($WR=0$) a **8001**: Il chip segue l'automa (va in Mode, poi in Control).
- Se leggi ($RD=0$) a **8001**: Il chip ignora l'automa e ti collega direttamente allo **Status Register**.



Indirizzo	C/D ⁻	Operazione CPU	Registro coinvolto nella USART
8000	0	Scrittura WR=0	Transmit Data Buffer (invio dato)
8000	0	Lettura DR=0	Receive Data Buffer (ricezione dato)
8001	1	Lettura RD=0	Status Register
8001	1	Scrittura WR=0	L'Automa: prima Mode , poi Control

Ora vediamo nei 3 registri di “controllo” i bit di cosa si occupano. Anche qui impararli a memoria non ha senso, ma bene o male qualcosa si può ricordare.

Status Register:

- **S₇->DSR attivo:** Questo bit è lo specchio fedele del pin fisico **DSR*** che si trova nel blocco **Modem Control**. Se è 0, significa che il dispositivo esterno è spento o scollegato. Il fatto che sia negato non è importante. Quando diventa 0 viene negato e noi lo vediamo sempre come 1. Se non fosse negato vedremo direttamente 1.
- **S₆->bit di sincronismo rilevati**
- **S₅->errore di framing**
- **S₄->errore di overrun**
- **S₃->errore di parità**
- **S₂->TxEMPTY**
- **S₁->RxRDY**
- **S₀->TxRDY**

I tre bit s₅,s₄,s₃ sono per i 3 errori. S₀ e S₁ sono lo specchio dei pin fisici, possono anche essere collegati alle interruzioni.

S₂ è l'informazione di stato per capire se è stato inviato tutto

Perché è fondamentale per il codice?

Quando ricevi l'interruzione su **RxRDY**, il tuo programma deve:

1. Andare all'indirizzo dispari (**8001**).
2. Fare una **Lettura** per caricare lo **Status Register**.
3. Controllare i bit **S₃, S₄, S₅**. Se sono a 0, allora può andare all'indirizzo pari (**8000**) a leggere il dato vero e proprio.
4. **S₇**: Questo bit è lo specchio fedele del pin fisico **DSR** che si trova nel blocco **Modem Control**.

Mode register: è il primo ad essere acceduto quando si parte dallo stato di reset in modalità scrittura

Bit	Funzione	Significato
M7	#sync bytes	0: 1 byte / 1: 2 byte (Solo in Sincrono)
M6	#stop bits	0: 1 bit / 1: 2 bit (Solo in Asincrono)
M5	Tipo Parità	0: Dispari / 1: Pari
M4	Abilita Parità	0: No / 1: Sì
M3-M2	#data bits	00: 5b / 01: 6b / 10: 7b / 11: 8b
M1	Non Usato	<i>Solitamente mantenuto a 0</i>
M0	Tipo Transm.	0: Sincrona / 1: Asincrona

Se la modalità M0 è asincrona significa che M7 verrà ignorato e si vede invece M6 per sapere quanti bit di stop ci sono. Se invece la modalità è sincrona M0 allora si ignora M6 e si vede M7.

Se sei in modalità SINCRONA

In questo caso **M7 è il protagonista**. Poiché nella trasmissione sincrona non ci sono bit di Start e Stop per ogni byte, il ricevitore ha bisogno di un segnale per capire dove inizia il messaggio.

- **M7 = 0 (1 Byte di Sync):** Il chip cerca un solo carattere speciale (caricato nel registro SYNC 1) per sincronizzarsi.
- **M7 = 1 (2 Byte di Sync):** Il chip cerca una sequenza di due caratteri (SYNC 1 + SYNC 2).

I bit M3/M2 indicano da quanti bit il dato effettivo è composto.

M4 abilita la parità, se 0 allora M5 viene ignorato, se 1 allora si guarda M5 che dice se la parità è dispari o pari

CONTROL REGISTER

Bit	Funzione	Significato
C7	Ricerca Sincronizzazione	Entra in "Hunt Phase" (solo Sincrono)
C6	Internal Reset	FORZA RESET (Torna al MODE)
C5	Attiva RTS	Pin RTS a livello logico basso (attivo)
C4	Reset Errori	Azzera i flag PE, OE, FE nello Status
C3	Non Usato	-
C2	Attiva Ricevitore	Ricezione Abilitata
C1	Attiva DTR	Pin DTR a livello logico basso (attivo)
C0	Abilita Tx/Rx	Abilita la sezione di trasmissione/ricezione

Tramite i bit di controllo la CPU può attivare le line Tx e Rx. se =0 significa che da Tx non esce niente e da Rx non entra niente. Ora se mettiamo 1 però possiamo anche rimanere solo in trasmissione tramite il Bit C2. Quindi C0 abilita la logica generale.

C1 e C5 servono per l'handshacking. C6 è il bit che serve a fare un reset per tornare nel MODE.

C7 si usa solo in modalità sincrona e forza il blocco ricevitore a smettere di leggere i dati e a rimettersi a cercare i caratteri SYNC nella linea. È utile se la comunicazione si "sporca" e vuoi riagganciare il segnale da zero. In pratica tutto ciò che arriva non lo considera più come dato, ma aspetta di riconosce i caratteri SYNC! E SYNC2 (o solo SYNC1)

Il chip che sta dall'altro lato (il trasmettitore) non sa minimamente che tu hai attivato C7. Lui continuerà a mandare i suoi dati.

- Se l'altro lato non manda i caratteri SYNC, il tuo chip rimarrà bloccato in "Hunt Phase" per sempre, aspettando invano di agganciarsi.
- Per questo motivo, nei protocolli sincroni, il trasmettitore invia periodicamente dei byte SYNC proprio per permettere a chi riceve di "ritrovare la strada" se ha usato il comando C7.

Esempio di configurazione (ASIM_Usart_TxRx_tutto ISR)

M7	M6	M5	M4	M3	M2	M1	M0
0	1	0	1	1	1	0	1

- $M_7=0$ -> 0 sync bytes
- $M_6=1$ -> due stop bits
- $M_5=0$ -> parità dispari
- $M_4=1$ -> parità trasmessa
- $M_3M_2=11$ -> 8 data bits
- M_1 -> non usato
- $M_0=1$ -> trasmissione asincrona

C7	C6	C5	C4	C3	C2	C1	C0
0	0	1	0	0	0	1	1

- $C_7=0$ -> no sincronizzazione
- $C_6=0$ -> no reset
- $C_5=0$ -> attiva RTS (per handshaking)
- $C_4=0$ -> bit di errore non attivati
- C_3 -> non utilizzato
- $C_2=0$ -> attiva trasmettitore
- $C_1=1$ -> attiva DTR (per handshaking)
- $C_0=1$ -> abilita periferica

In questo esempio vediamo $M_0=1$ quindi modalità asincrona (M_7 viene ignorato). M_1 non si usa, $M_3M_2=11$ quindi dato di 8 bit, $M_4=1$ parità attiva, $M_5=0$ parità dispari, e $M_6=1$ dice 2 bit di stop

Quindi un ipotetico messaggio sarà

010100011100

Start | 8 bit dato | parità | due bit Stop

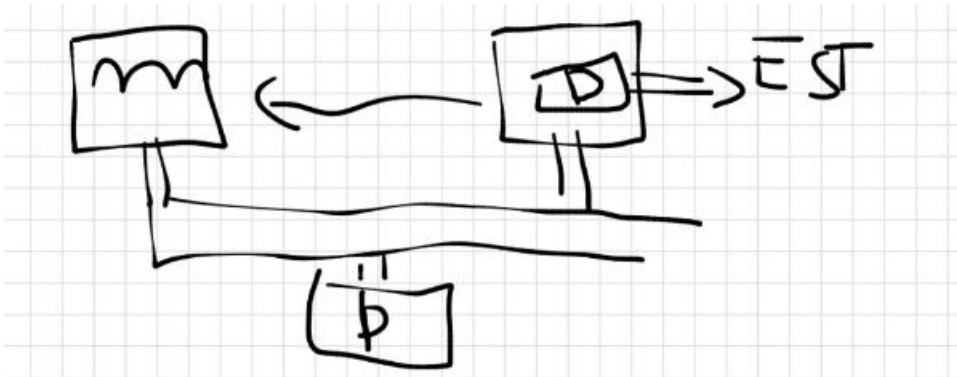
Avendo modalità asincrona nel momento in cui scriviamo il modo, il puntatore passa al registro di controllo.

$C_0=1$ Tx/Rx abilitate, $C_1=1$ DTR parte quindi inizia handshaking, $C_2=0$ ricevitore non attivo, C_7 viene ignorato, gli altri all'inizio non servono.

Lezione 10 Aprile

Se la periferica è parallela l'unico modo da settare è se i bit sono in entrata o in uscita, se l'interruzione si propaga o no, se i bit sono significativi quando va da 1 a 0 o da 0 a 1. Se andiamo sulla seriale la prima cosa da decidere se la modalità è sincrona o asincrona. La seconda cosa se è asincrona una serie di parametri legati al frame che viene trasmesso. Se invece è sincrona dobbiamo settare i parametri della sincronizzazione.

Tutti i parametri sono relativi alla tipologia della comunicazione.



Oggi vogliamo gestire dei dispositivi che si mettono in mezzo alla seriale /parallela e il processore e la memoria. Quindi non comunica con l'esterno ma sta dentro.

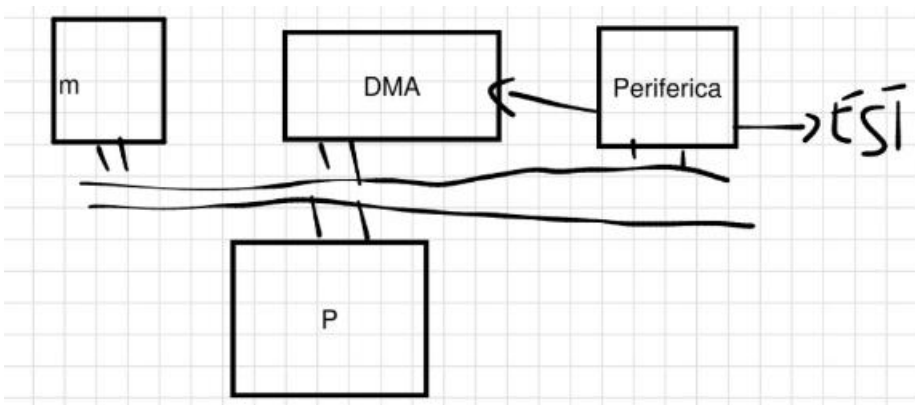
Modello con o senza dma. Se faccio il modello senza dma ho memoria, bus, processore e poi la periferica che va verso l'esterno. Questa periferica si collega al BUS, ed è in memoria. Al di là del segnale di handshaking, dopo aver letto il dato deve portarlo in memoria. Move.B Dato, MEM

Dove il dato sta nella periferica

Il processore deve scrivere nella memoria. Servono 3 informazioni: deve dare il dato, l'indirizzo e i segnali di read/write.

Prendi il dato, scrivilo in memoria. È un'operazione in cui il dato sappiamo dove si trova. Il processore comunque perde del tempo perché deve fare delle operazioni di scrittura in memoria. Non abbiamo conflitti.

Se abbiamo il DMA vogliamo che il processore fosse svincolato da tutte le azioni, come interruzioni e non vogliamo che faccia sempre lui le operazioni idiote di scrivere in memoria questo dato.



Stiamo chiedendo ad un altro oggetto di fare quest'azione al posto del proc. Deve garantire che i segnali che prima andavano al processore non arrivano più, altrimenti il proc si interrompe. Quando il carattere è pronto non vogliamo interrompere il processore, ma vogliamo che il dma faccia queste 3 operazioni al posto suo. E vogliamo che lo faccia per molti caratteri, quando c'è un messaggio.

Quale può essere l'equivalente della parallela e seriale per quanto riguarda il modo? Cosa gli dobbiamo dire per poter funzionare? Capiamo il modello.

Per poter operare deve sapere qual è l'indirizzo dove deve scrivere. Prima lo dava il processore ora lo dovrà dare il DMA. Se non ha l'indirizzo come fa a scrivere. Non solo deve prendersi il segnale per chiudere il

protocollo. Il DMA deve sapere pure quanti caratteri deve dare, se non lo sa non può operare. Ogni volta che il DMA scrive grazie al numero di caratteri incrementa l'indirizzo. Quando arriva alla fine potrebbe generare un'interruzione verso il processore per dire ho finito. Il DMA può avere più canali perché se devo fare un trasferimento di dati da una memoria a una memoria ne ho bisogno di 2.

Il DMA è attivato dal processore, quando va a scrivere può andare in conflitto con il processore che scrive. Nel frattempo il processore può fare altre cose. Deve gestire un arbitraggio del BUS. Uno solo deve andare sul BUS.

Serve la terza cosa fondamentale: sapere qual è la modalità di arbitraggio. Le due più importanti sono le due estreme: ogni volta che vado in memoria, devo fare una richiesta al processore, ma non crea un'interruzione senno sarebbe assurdo. È una cosa hardware. Posso andare in memoria? Sì e ci va, ma il proc può continuare a fare i suoi calcoli. Un altro modello estremo è te lo chiedo una sola volta e mi impossesso del bus per tutto il tempo che scrivo il messaggio. Se una periferica è lenta non ha senso fare così. Se devo fare un trasferimento memoria-memoria che è veloce può avere senso usare il DMA. Se devo fare un trasferimento con una seriale che è lenta non ha senso usare questa versione blocked.

Il DMA è importante per una cosa fondamentale. Tutte le cose che facciamo sono messaggi. La rete invia messaggi, etc. sono sequenze di caratteri.

La periferica parallela ha un sistema di regime e transitorio. Il transitorio è leggere in modo che si appaia in modo da fare resetting. A regime funziona perfettamente.

Alla periferica il modo di farlo deve dare il processore. Il processore setta la periferica, setta il DMA.

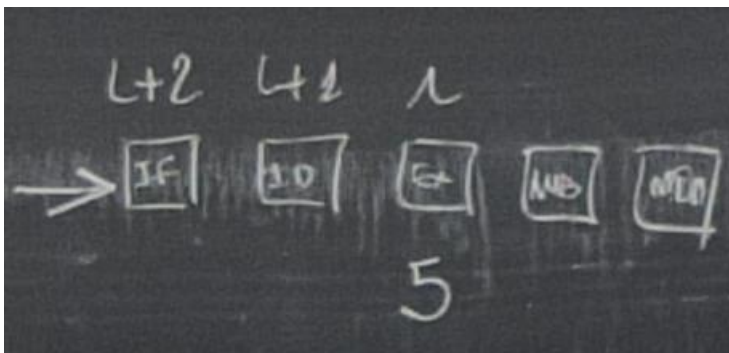
Lo schedario ha tutto. Ci chiede in fase di configurazione la seriale, la parallela, lo vuoi con DMA o senza DMA. Il momento della configurazione decide cosa ci deve stare. Ci sarà una funzione che è initDMA e initPERIFERICA. Ci dà delle funzioni già date.

Quando la CPU esegue un'istruzione, attraversa diverse fasi (Fetch, Decode, Execute, ecc.).
Se il processore sta eseguendo un'operazione matematica pura tra i suoi registri interni non sta utilizzando il bus dei dati e degli indirizzi in quel preciso ciclo di clock.
Il DMA approfitta proprio di questi momenti, o forza una brevissima pausa hardware, senza toccare il flusso logico del programma.

Il DMA fa un ciclo rubato. Se il processore sta facendo dei calcoli, è ovvio che il dma chiede il bus, ma il processore in quel momento sta facendo l'operazione. Non c'era conflittualità perché il segnale è esterno.

Quando il DMA finisce però lo avvisa con interrupt.

Il processore faceva delle fasi separate. Tutto l'hardware serviva in fasi differenti, ma facevano un'istruzione alla volta. Il PC serviva all'inizio, poi serviva per prelevare la successiva. Oggi i processori moderni hanno un'architettura diversa. Nel caso del MIPS ha 5 stadi. Uno prende solo istruzioni, uno decodifica solo, uno le esegue. Contemporaneamente fa 5 istruzioni. Sto eseguendo l'istruzione i, sto decodificando l'istruzione i+1 e sto prendendo l'istruzione i+2.



Questo sistema ha un problema. Il rate di accesso alla memoria è grande perché c'è un'unità che legge solo dalla memoria.

Quindi questi sistemi che studiamo noi può stare in qualche cosa semplice, ma ormai non esiste più nei sistemi di calcolo.

Oggi sono tutti sistemi pipelined.

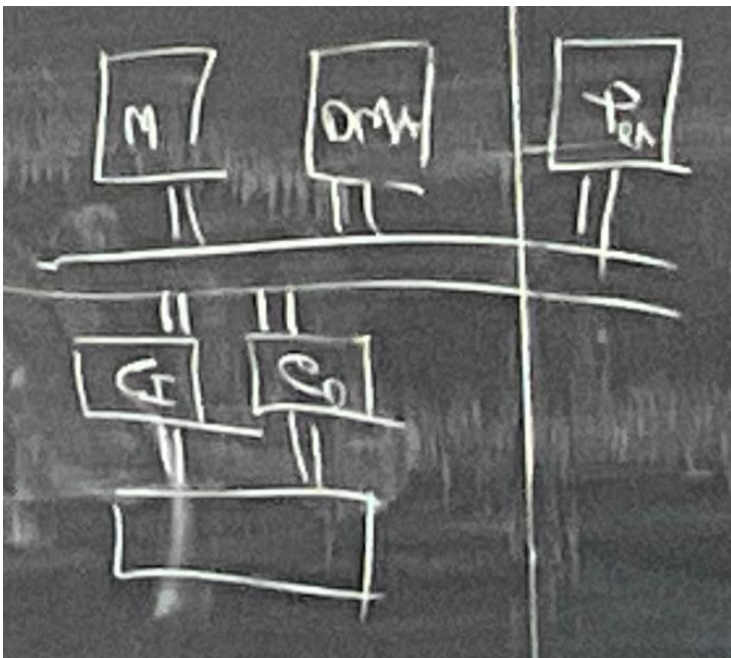
Parallelismo di unità: uso 2 processori e devo capire come farli parlare. So cose separate.

Parallelismo di nodo: all'interno di un nodo non elaboro una sola istruzione, ma di più.

Esiste un problema molto forte su questa tipologia di modello. C'è una memoria istruzioni e una memoria dati. Non ho una sola memoria perché uno stadio vuole leggere e uno vuole scrivere fanno a capate. Quindi ho due memorie. Ho bisogno anche che queste memorie siano molto veloci.

Le memorie dei sistemi che studiamo noi non riescono a stare dietro al processore perché sono molto lente, ma grandi. Voglio memorie veloci e grandi. È impossibile. Devo fare memorie veloci e per forza piccole.

Il processore ha due memorie veloci sono chiamate memorie cache. Essendo piccole dobbiamo capire come fanno ad avere le fotocopie delle memorie grandi. Dobbiamo aggiornare queste fotocopie. Questo modello lavora sempre con le cache, ma se non ha il dato devo andare in memoria. E quando accade so dolori di pancia. Spero che accada poche volte, ma sicuro accade. Se facciamo questo modello, l'utilizzo del DMA mi dà meno fastidio. Perché il processore lavora sulle cache, mentre il DMA sulla memoria. Il problema è quando non ho un dato in cache e devo andare in memoria, e il DMA va in memoria. Ho spostato il problema, ma accadrà pochissime volte. Questo modello ha un sacco di problemi di coerenza della memoria. Ho una gerarchia di memorie. Il processore lavora con memorie gerarchicamente più veloci.



DMA

Il dispositivo Direct Memory Access (DMA) è un dispositivo in grado di operare da master di bus, in grado di gestire cicli bus di trasferimento dati (in sostituzione del processore). Esso viene utilizzato per aumentare le prestazioni di un sistema di calcolo gestendo, in sostituzione del processore ma in modo più efficiente, le operazioni di trasferimento, consentendo, allo stesso tempo, al processore di operare in parallelo. La maggiore efficienza è dovuta al fatto che mentre un processore per eseguire un trasferimento deve eseguire

un'istruzione (ad es. un codice di move) attraverso una fase di fetch ed execute, un DMA gestisce il trasferimento utilizzando Hw specializzato appositamente progettato per realizzare tale unica funzione.

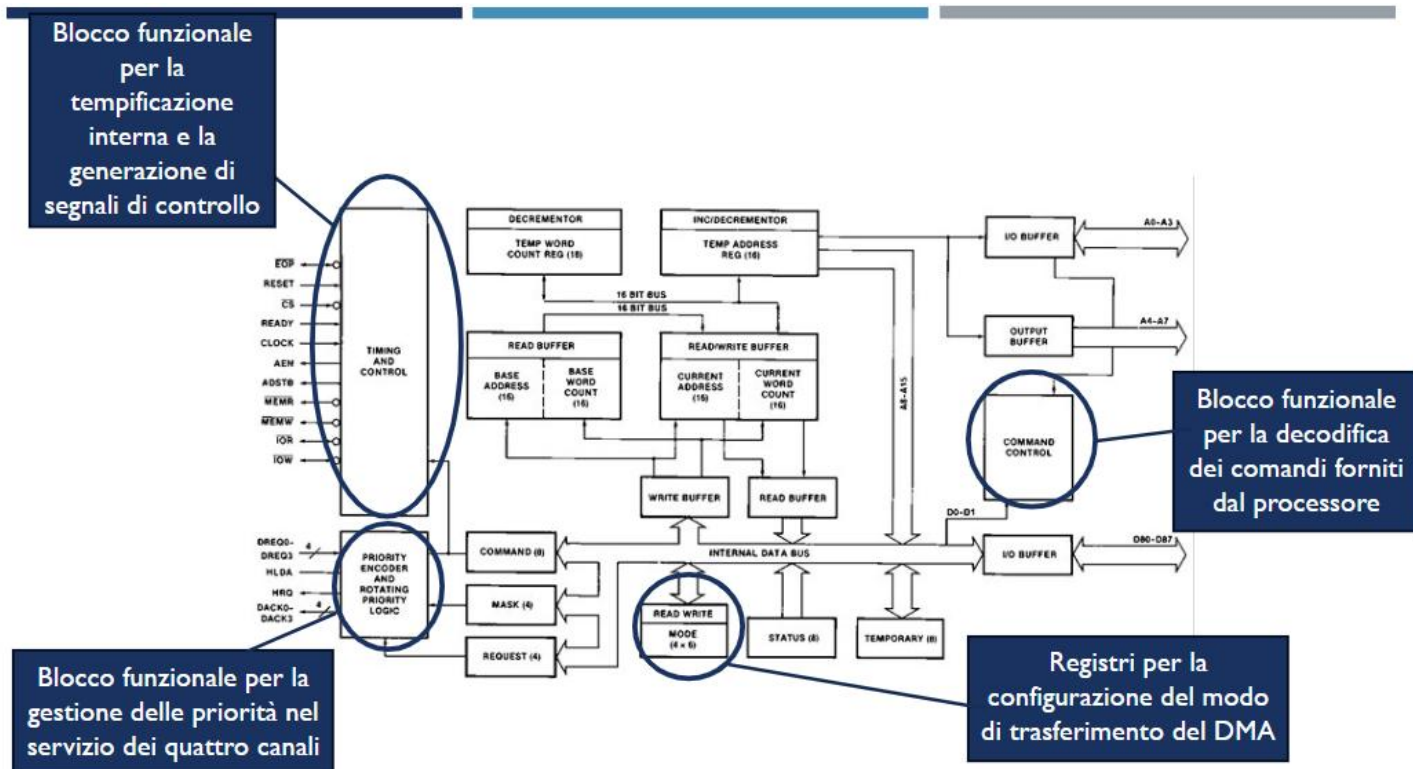
Funzionamento in breve

In un'operazione di DMA il processore attiva la linea R/W per indicare la lettura o scrittura, pone sul bus dati l'indirizzo fisico del dispositivo di I/O interessato e, successivamente, l'indirizzo iniziale in memoria del blocco dati coinvolto nell'operazione (da dove leggere o dove scrivere i dati) e la quantità di dati da trasferire, avviando in tal modo l'operazione di trasferimento del blocco dati. Il processore può, a questo punto, eseguire altri codici operativi (sempre che non si creino conflitti di accesso sul bus) mentre il controllore DMA si occupa del trasferimento dei dati colloquiando direttamente con la memoria centrale. Alla fine del trasferimento del blocco il controllore DMA invia un interrupt alla CPU al fine di avviare le operazioni di chiusura dell'operazione.

Quando grandi volumi di dati devono essere trasferiti, si ricorre ad un meccanismo più efficiente, quello basato sul DMA in grado di operare parallelamente al processore e gestire, mediante hardware specializzato le operazioni di trasferimento. Un dispositivo DMA controller è, infatti, capace, opportunamente programmato, di diventare il master del bus e supervisionare un trasferimento tra memoria e memoria o un'interfaccia periferica o una memoria di massa, senza intervento del processore. Mentre esegue un trasferimento, il DMA pone gli indirizzi di memoria sul bus e spedisce e riceve i segnali necessari per l'effettuazione di operazioni di lettura e scrittura in memoria e sulle interfacce periferiche. Lo scopo di un DMA controller è, quindi, quello di realizzare una sequenza di trasferimenti mediante l'uso di cicli di bus sottratti al processore.

- **La periferica dispone di 4 canali per il collegamento con 4 dispositivi diversi**
- **Il trasferimento del DMA può avvenire secondo quattro modalità:**
 - **single (una parola per volta):**
 - **block (trasferimento di un intero blocco)**
 - **demand (block + ripresa dal punto di sospensione)**
 - **cascade (collegamento in cascata di più DMA)**

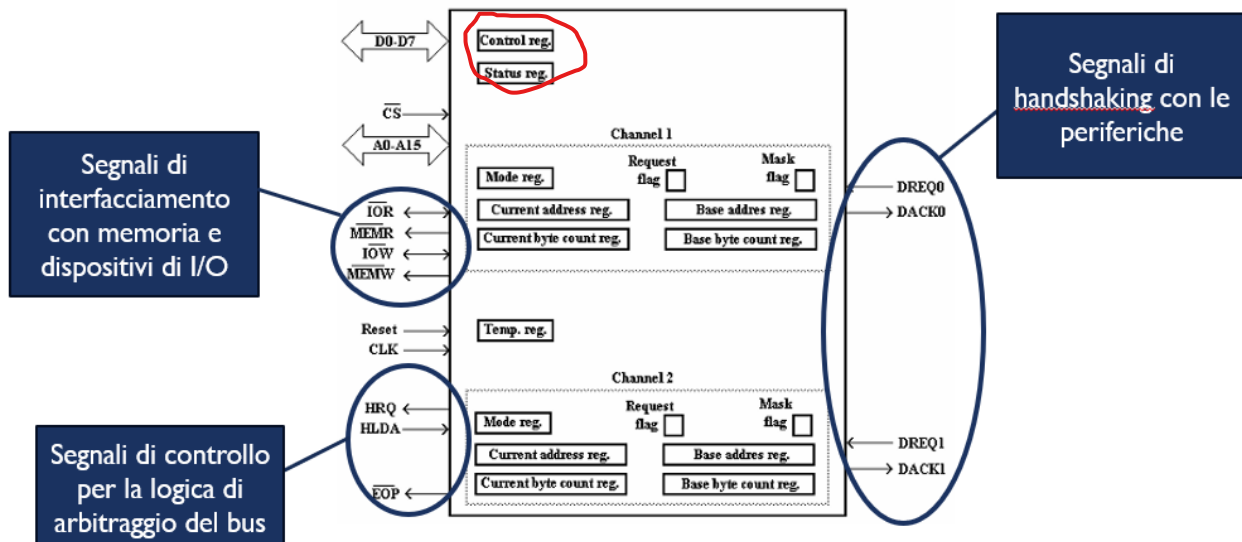
Architettura interna



In ASIM è implementata con un'architettura semplificata: ci sono 2 canali non 4.

- Ogni canale possiede 5 registri e 2 flag:
 - **Registro di modo**
 - **Registro base address**: in fase di configurazione. Indirizzo di memoria dal quale far partire il trasferimento
 - **Registro base byte count**: contatore che va a decremento
 - **Registro current address**: quale indirizzo è attualmente interessato dal trasferimento
 - **Registro current byte count**: serve per controllare quanti byte ho già letto tramite trasferimento con DMA
 - **Request flag**: avviare la comunicazione con il DMA tramite software. Nel trasferimento memoria- memoria non è necessario
 - **Mask flag**: mascherare le interruzioni di quel canale.

Il registro di controllo e il registro di stato sono condivisi dai due canali. Poi c'è un registro temporaneo che viene utilizzato nello scambio memoria- memoria



Questi segnali noi non li vediamo quando configuriamo la periferica.

Bus Arbitration Control: segnale del processore per la gestione del BUS

- !BR (!bus request-in): segnale che indica al processore la richiesta di bus da parte di un dispositivo. Tale segnale va posto in or di connessione se a richiedere il bus sono più dispositivi;
- !BG (!bus grant-out): segnale che indica a tutti i device richiedenti il bus che il processore rilascerà il bus alla fine del ciclo corrente;
- !BGACK (!bus grant acknowledge-in): segnale che indica che un device è divenuto il bus master e lo sta gestendo.

Il DMA acquisisce il controllo del bus tramite il seguente processo:

1. Il DMA richiede il possesso del bus al processore tramite il segnale bus request (bus request-in, BR)
2. Il processore fornisce l'assenso alla gestione del bus (bus grant-out, BG)
3. Il dispositivo che ha richiesto il bus fornisce l'ack al processore (bus grant acknowledge-in, BGACK)
4. Il processore viene scollegato elettricamente dal bus indirizzi

Comunicazione con un device

- Il DMA si interfaccia con le periferiche di I/O tramite i segnali DREQ e DACK
- DREQ è inoltrato da un dispositivo di I/O per richiedere la gestione al DMA di trasferire dati
- DACK è utilizzato dal DMA per informare il dispositivo che è stato selezionato per un trasferimento
 - DACK è fornito una volta che il DMA ha acquisito il controllo del bus indirizzi

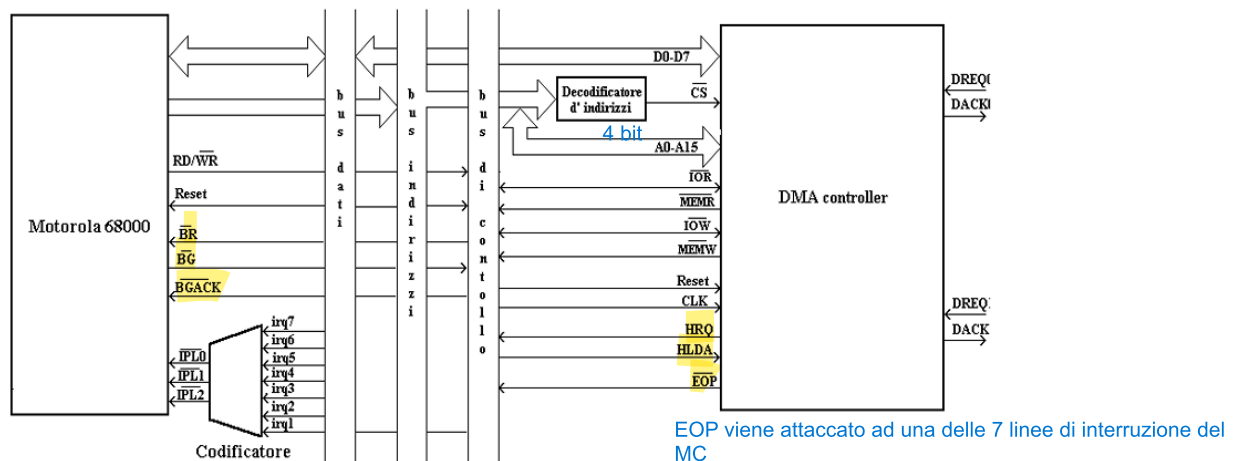
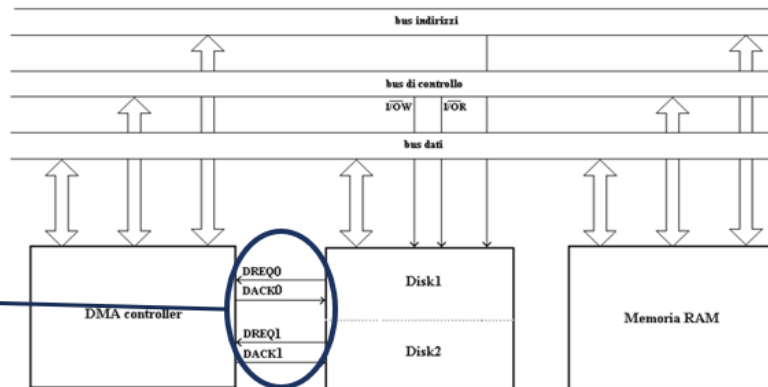


Fig. Errore. Nel documento non esiste testo dello stile specificato.-1 Schema di collegamento del DMA

1) Collegamento al processore

Le linee di collegamento al processore, sono mostrate sulla sinistra dello schema di Fig.1 Schema di collegamento del DMA. In particolare, **D0-D7 sono 8 linee che vanno connesse al bus dati per il trasferimento dei dati da e verso il componente;** CS è utilizzata per la selezione del dispositivo. Il controller, per eseguire un trasferimento, deve attivare i segnali necessari per l'effettuazione di operazioni di lettura e scrittura in memoria (MEMR e MEMW) e sulle interfacce periferiche (IOR e IOW). Sulle linee CLK e Reset

arrivano rispettivamente il segnale dal generatore di clock e il segnale di reset che esegue il posizionamento al valore 0 di tutti i registri del controllore.

La linea **HRQ (Hold Request) → BR (Bus Request)** è adoperata per spedire una richiesta di controllo del sistema bus. Essa normalmente è applicata all'ingresso HOLD della CPU. Un segnale, in arrivo dalla CPU, sulla linea **HLDA (Hold Ack) ← BG (Bus Grant)** indica che è stato acquisito il sistema bus. La linea **d'interruzione EOP** trasmette al processore, o a un eventuale gestore delle interruzioni, un'interruzione per avvisare che il trasferimento di un blocco di memoria è stato completato.

2) Collegamento ad un device

Sulla destra dello schema in Fig.1 Schema di collegamento del DMA, sono mostrate le linee fisiche che collegano il DMA controller alle periferiche. In particolare, **DREQ0 e DREQ1** sono le linee di richieste, usate dalle periferiche collegate ai rispettivi canali, per ottenere dei cicli DMA. Le richieste che arrivano su DREQ0 hanno precedenza su quelle che arrivano su DREQ1. Le linee **DACK1 e DACK2** informano la periferica, connessa a quel canale, che è stata selezionata per un ciclo DMA. Queste linee si comportano, nei confronti dei componenti periferici che richiedono questo servizio, come un "chip select".

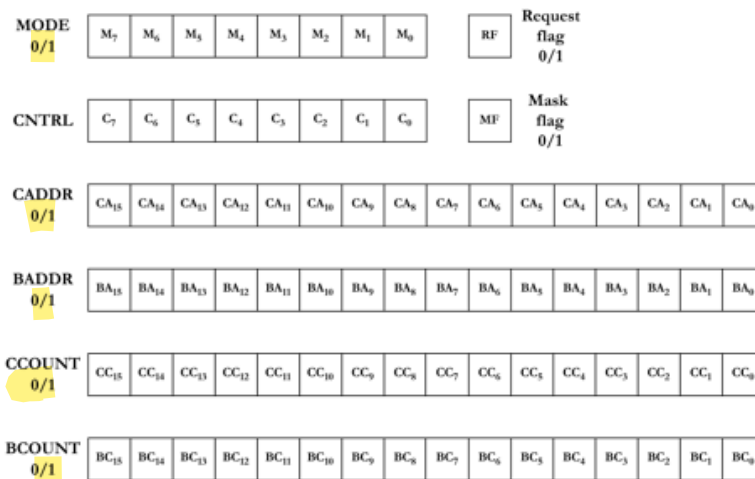
Modello di programmazione

Consideriamo il canale 0 del DMA

- **Il registro CADDR0 (indirizzo corrente del trasferimento)** contiene l'indirizzo della locazione di memoria interessata dal trasferimento
 - Accedibile in lettura/scrittura, indirizzo relativo \$0. Ci fa visualizzare l'indirizzo che attualmente è interessato nel trasferimento
- **Il registro BADDR0 (indirizzo base del trasferimento)** contiene l'indirizzo iniziale di CADDR0
 - È importante in configurazione per capire da dove deve partire il trasferimento. Quando tu scrivi il valore in CADDR0, l'hardware del chip fa una copia ombra e lo salva automaticamente anche in BADDR0.
- **Il registro CCOUNT0 (conteggio corrente)** contiene il numero di byte da trasferire
 - Accedibile in lettura/scrittura, indirizzo relativo \$1: è un contatore alla rovescia. Ad ogni byte trasferito si decrementa.
- **Il registro BCOUNT0 (conteggio di base)** contiene il numero di byte iniziale da trasferire
 - Accedibile in scrittura automaticamente insieme a CCOUNT0 come per BADDR0
- **Il registro MODE0 contiene informazioni sul modo di funzionamento del canale 0**
 - Accedibile in scrittura, indirizzo relativo \$B
 - Se il byte di configurazione ha bit meno significativo pari a 0, seleziona il canale 0
- **Il registro CNTRL è suddiviso in 2 parti: i 4 bit meno significativi indicano lo stato del componente, quelli più significativi il bit di controllo**
 - Accedibile in scrittura e lettura, indirizzo relativo \$8
- **Il flag RF0 indica le richieste software verso il DMA**

- Accedibile in scrittura, indirizzo relativo \$9
- Se il byte di configurazione ha bit meno significativo pari a 0, seleziona il canale 0, altrimenti canale 1
- Bisogna alzare il bit numero 3 indicativo del flag RFO
- Il flag MF0 maschera le richieste del canale
 - Accedibile in scrittura, indirizzo relativo \$A
 - Se il byte di configurazione ha bit meno significativo pari a 0, seleziona il canale 0
 - Il valore del flag dev'essere posto al bit numero 2 del dato

Tutte le periferiche hanno in comune che devo scegliere il modo, devono comunicare con il bus.



l'ultimo bit indica se è il canale 0 o il canale 1

- RF: avvio del DMA
- MF: mascheramento del DMA

- CADDR0/1
 - CA_{15..0}->indirizzo corrente della locazione interessata dal trasferimento
- BADDR0/1
 - BA_{15..0}->indirizzo di base dell'area di memoria interessata dal trasferimento
- CCOUNT0/1
 - CC_{15..0}->numero di byte da trasferire
- BCOUNT0/1
 - CC_{15..0}->numero di byte totali da trasferire
- CNTRL register
 - C₇->abilitazione DMA
 - C₆->vincolo trasferimento memoria-memoria
 - C₅->trasferimento memoria-memoria
 - C₄->non utilizzato
 - C₃->richiesta inoltrata al canale 1 (DREQ1)
 - C₂->richiesta inoltrata al canale 0 (DREQ0)
 - C₁->termine conteggio per il canale 1
 - C₀->termine conteggio per il canale 0

C5 abilita il trasferimento memoria-memoria, mentre C6 serve per abilitare un vincolo del trasferimento memoria memoria. non ci interessa.

RF e MF sono semplici bit.

I 4 registri sono a 16 bit e sono le informazioni dette precedentemente.

Il registro di controllo e di modo sono a 8 bit (come per le altre periferiche)

Il registro di controllo che tipicamente serve per definire se stiamo abilitando o meno il DMA a trasferire.

Oppure definire se il trasferimento è memoria-memoria o memoria-periferica.

- **MODE0/1**
 - **M₇->modo del trasferimento**
 - 0: single, 1: block
 - **M₆->non utilizzato**
 - **M₅->modalità di incremento/decremento di CADDR**
 - 1: decremento, 0: incremento
 - **M₄->autoinizializzazione**
 - **M₃->direzione del trasferimento**
 - 0: memoria-interfaccia, 1: interfaccia-memoria
 - **M_{2,1}->non utilizzati**
 - **M₀->selezione del canale**

La differenza tra single e block è un meccanismo attivato a basso livello. single vuol dire che ad ogni trasferimento il DMA si accerta che può acquisire il bus. Mentre block si assicura che l'acquisizione avvenga una sola volta e il processore non può intervenire.

M5 dice se l'indirizzo deve essere incrementato o decrementato. M3 anche è importante e serve per dire se il trasferimento è memoria-interfaccia o interfaccia-memoria.

RESET: riporta il componente nello stato iniziale, azzerando tutti i registri; per attivarlo basta accedere in scrittura all'indirizzo relativo \$D;

L'operazione di trasferimento inizia quando la periferica collegata al canale 0 spedisce una richiesta al DMA e quest'ultimo l'accetta. La stessa richiesta, invece che mediante un segnale hardware, potrebbe anche realizzarsi in software mediante l'uso del flag RF0 (o RF1 per l'altro canale).

Completato un ciclo di bus, questo viene rilasciato al processore se il valore del bit 7 del registro MODE è zero (trasferimento a singolo byte); se posto a 1, si susseguono, invece, tutti gli altri cicli necessari al trasferimento dell'intero blocco.

Al termine del trasferimento di un blocco, cioè quando il valore CCOUNT diventa nullo, sono eseguite le seguenti operazioni:

- viene cancellato il flag RF (è il flag dove un processore può indirizzare le sue richieste ed hanno gli stessi effetti di quelle che arrivano dai device),
- è azzerato il bit 2 o 3 di CNTRL (DREQ) a seconda che si sia utilizzato rispettivamente il canale zero o uno,

- è posto ad uno il bit 0 o 1 di CNTRL (bit di fine conteggio) rispettivamente per il canale zero e uno,
- infine viene inviata un'interruzione del tipo specificato in COM2 al gestore delle interruzioni specificato in COM3 (EOP)

COM1: definisce l'identificatore della CPU alla quale richiedere il controllo del bus mediante un handshaking (si simula il collegamento logico delle linee HRQ e HLDA).

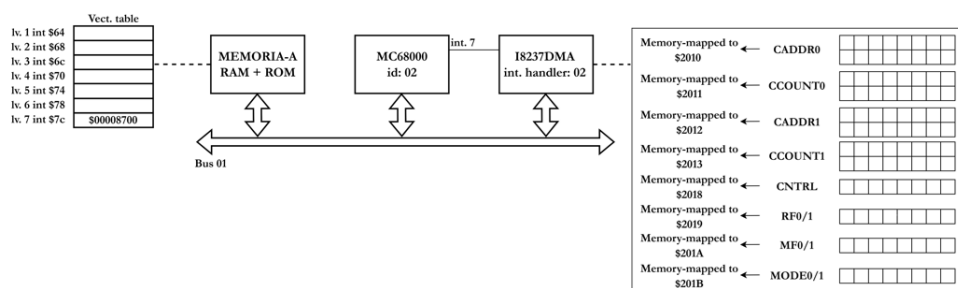
COM2: specifica del tipo di interruzione trasmessa al gestore delle interruzioni (CPU o PIC), indicato in COM3, alla fine del conteggio di un trasferimento di un canale. Delle quattro cifre esadecimali che definiscono COM2 la meno significativa individua la linea d'interruzione, la seconda definisce la priorità e le due più significative specificano il vector number da trasmettere al processore che gestisce l'interruzione. Se le interruzioni sono gestite da un PIC, queste due cifre non devono essere specificate. I due parametri COM2 e COM3 determinano, quindi, la connessione logica della linea d'interruzioni EOP al processore o al PIC.

COM4: le 2 cifre meno significative e le 2 più significative indicano, rispettivamente, l'identificatore della periferica collegata al canale 0 e al canale 1. Specificando il valore in COM4 si definisce, di fatto, a quali componenti collegare logicamente le coppie di linee (DREQ0, DACK0) e (DREQ1, DACK1).

Name	I8237DMA
Type	Device
Identif	Intero (\$01..\$FF) che identifica univocamente l'oggetto in una configurazione
Address1	Indirizzo base del dispositivo
Address2	Indirizzo base+\$F (DMA occupa 16 locazioni di memoria)
BUS	Ident. del bus a cui il dispositivo è connesso
COM1	Ident. della CPU master per il trasferimento
COM2	vettore (b15-b8) priorità (b7-b4) linea Int (b3-b0)
COM3	Ident. Gestore interruzioni
COM4	bit0-3 e bit4-7 ident. dispositivi connessi al canale 0 e 1

Configurazione memoria-memoria

- La configurazione istanzia un singolo sistema dove il DMA acquisisce il controllo del bus per effettuare il trasferimento di alcuni dati a partire dall'indirizzo origine \$00009500



Nel trasferimento **memoria-memoria**, il DMA non sta lavorando con una periferica esterna (che ha i suoi tempi), ma sta usando se stesso per spostare dati da un punto A della RAM a un punto B della RAM. Per farlo, "affitta" due canali contemporaneamente per compiti diversi:

1. **Canale 0:** Diventa il lettore (Sorgente).
2. **Canale 1:** Diventa lo scrittore (Destinazione).

Siccome il canale 0 è usato per leggere dalla memoria non ha bisogno di configurare il registro di conteggio dei byte. Quello si usa quando trasferiamo cioè quando scriviamo.

In un trasferimento da memoria a memoria il byte da spostare viene letto dalla locazione sorgente e spostato nel registro temporaneo, **TEMP** e da questo spostato nella locazione di memoria di destinazione. Questo registro può essere solo letto ed ha indirizzo relativo pari a \$D.

*inizializzazione del device DMA

```

        MOVE.W    #dma,A0                ;Carica in A0 l'indirizzo base del DMA
        MOVE.W    #origine,caddr0(A0)    ;Carica l'indirizzo base del blocco sorgente nel
*                                           registro indirizzo del canale 0
        MOVE.W    #destinazione,caddr1(A0) ;Carica l'indirizzo base del blocco destinazione nel
*                                           registro indirizzo del canale 1
        MOVE.B    #nbyte,ccount1(A0)     ;Carica il numero di byte da spostare nel registro
*                                           conteggio
        MOVE.B    #$90,mode(A0)          ;CANALE 0: setta il trasferimento da memoria ad
*                                           interfaccia, si autoinizializzazione dei registri
*                                           addr e count dopo trasferimento, incremento del
*                                           registro indirizzo,trasferimento in modalità BLOCK
        MOVE.B    #$91,mode(A0)          ;CANALE 1: setta il trasferimento da memoria ad
*                                           interfaccia, si autoinizializzazione dei registri
*                                           addr e count dopo trasferimento, incremento del
*                                           registro indirizzo,trasferimento in modalità BLOCK
        MOVE.B    #$A0,cntrl(A0)         ;CNTRL=1010|0000: abilita trasmissione mem-to-mem e
*                                           abilita il DMA controller (automaticamente viene
*                                           spedita la richiesta di bus al processore)
loop    JMP      loop                    ;Loop di attesa del trasferimento, simula un impegno
*                                           fittizio del processore

```

Il DMA occupa un blocco di indirizzi nella memoria della CPU. Ogni registro ha un "indirizzo relativo" (offset) rispetto all'indirizzo base del DMA.

Il registro MODE non è un "vassoio" unico

Sebbene l'indirizzo sia lo stesso (\$B), all'interno del chip il registro di modo è sdoppiato.

- Quando invii un byte, il chip non si limita a salvarlo; guarda il **bit0** del dato che hai appena spedito. Funge da indirizzo interno.
1. **MOVE.B #\$90, mode(A0):** Il dato è 10010000. Il DMA vede b0=0, capisce che deve configurare il **Canale 0** e salva le impostazioni nel "cassetto" del Canale 0.
 2. **MOVE.B #\$91, mode(A0):** Il dato è 10010001. Il DMA vede b0=1, capisce che deve configurare il **Canale 1** e salva le impostazioni nel "cassetto" del Canale 1.

10010000 del MODO mette modo di trasferimento blocco e poi M4=1 modalità auto inizializzazione, poi modalità incremento. La modalità è memoria-interfaccia.

Il DMA "pensa" sempre di parlare con un'interfaccia

Il controller DMA è nato per spostare dati tra RAM e Periferiche (I/O). Non è stato progettato nativamente per il memoria-memoria; questa funzione è stata aggiunta "sfruttando" l'architettura esistente.

- Per il DMA, un trasferimento avviene sempre tra un **Indirizzo** (RAM) e un **Segnale di DACK** (interfaccia).
- Quando abiliti la modalità memoria-memoria (C5=1 nel registro di controllo), il chip crea un ponte interno: usa il Canale 0 come se fosse una periferica che "legge" e il Canale 1 come se fosse una periferica che "scrive".

□ **Canale 0 (Sorgente):** Viene configurato come "Memoria ad Interfaccia" perché deve **leggere** dalla memoria. Il dato viene prelevato dalla RAM (all'indirizzo caddr0) e "parcheggiato" in un registro temporaneo interno al chip DMA.

□ **Canale 1 (Destinazione):** Viene configurato anch'esso come "Memoria ad Interfaccia" (ma con il bit di direzione invertito o gestito dal ponte C5) perché deve **scrivere** verso la memoria. Prende il dato dal registro temporaneo e lo "spara" nella RAM (all'indirizzo caddr1. Infatti solo per questo canale stabiliamo il numero di byte da trasferire perché è colui che trasferisce.

In ASIM nel file di configurazione possiamo vedere delle cose importanti. Intanto il BUS a cui è collegato che è quello della memoria. Poi indirizzo di partenza e fine per accedere ai registri memory mapped, e poi a che livello di interruzione (0007 su COM2, quindi livello 7)

Nel codice abbiamo una serie di MACRO per definire gli spiazziamenti per accedere ai registri.

Per la ISR dobbiamo metterla in memoria. Se abbiamo già la memoria la carichiamo con LOAD Memory. Per vedere se è caricata basta fare showLOc, scriviamo l'indirizzo e vediamo se è caricata. poichè è la modalità block, quando termina l'intero blocco saltiamo all'interruzione di fine trasferimento che ripristina lo stato del DMA

```
*      INTERRUZIONE DI FINE TRASFERIMENTO      *
*      questa interruzione è generata dal segnale EOP quando il conteggio dei byte da trasferire si azzerà.
*      L'interruzione deve essere sempre prevista: in questo caso specifico l'interruzione resetta il DMA
*
      ORG $8700

int7      MOVE.L  A0,-(A7)                ;Salva il contesto
          MOVE.W  #dma,A0
          MOVE.B  #0,reset(A0)            ;Resetta il DMA
          MOVE.L  (A7)+,A0                ;Ripristina i registri utilizzati
          RTE

          end                               start
```

Reset è 13(A0)

Il programma quando parte inizialmente salva in A0 l'indirizzo del DMA che si può vedere dalla configurazione nell'indirizzo di start e poi dobbiamo caricare i registri: indirizzo di partenza, indirizzo di destinazione, numero di byte da trasferire (nel canale 1 lo mettiamo), registri di modo e di controllo per abilitare il DMA.

Ora il processore fa LOOP JMP LOOP cioè non si interessa di cosa succede.

Il processore ha finito, vediamo qualcosa del DMA? Assolutamente niente, lavorano in parallelo, ormai il nostro lo abbiamo fatto, ora se la vede il DMA, decrementa il contatore e trasferisce, quando ha finito manda l'interruzione e la gestiamo resettando il DMA.

Al massimo posso vedere che nella memoria piano piano vengono spostati gli elementi.

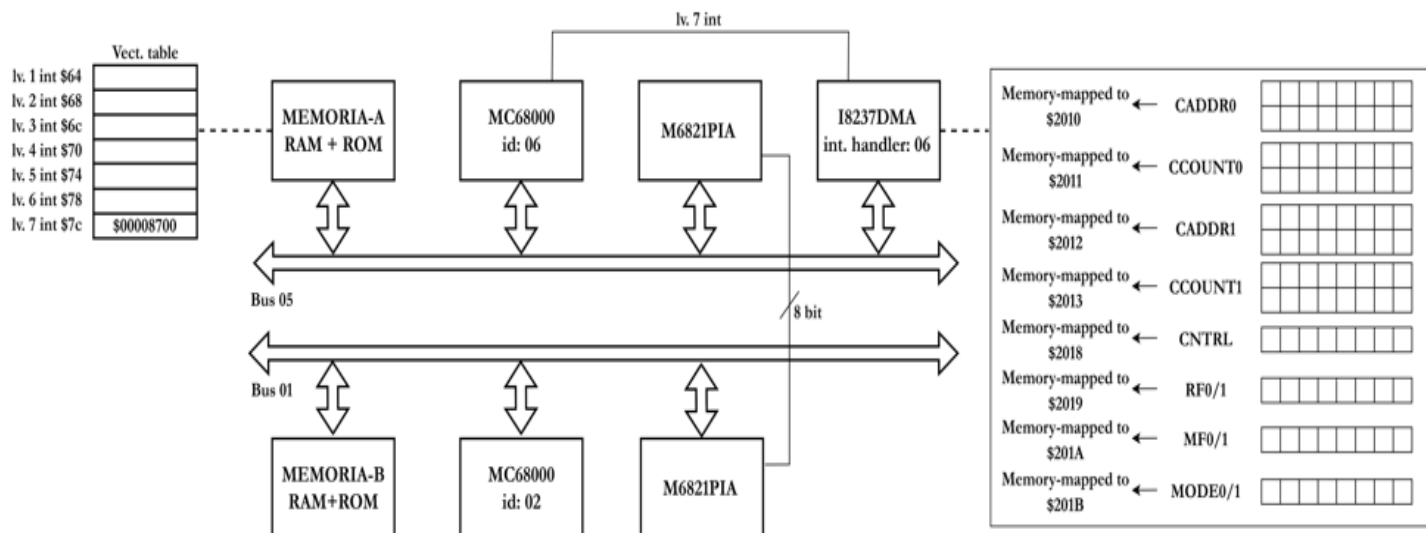
Se ci posizioniamo nella locazione in cui avviene il trasferimento vediamo che piano piano i byte vengono trasferiti e il contatore viene decrementato, senza che il processore intervenga.

Quando attivo il trasferimento sotto interrupt, il primo carattere devo trasferirlo io, quelli successivi vengono trasferiti sotto interrupt. Nel caso del DMA il primo carattere non lo devo trasferire io, perché già dal primo carattere il DMA è attivo, non ho bisogno di attivare il canale.

Tornando al discorso sulla cache, se io non trovo un dato nella cache e lo devo portare dalla memoria alla cache, perché non posso usare il DMA? Chi lo fa? Il processore o l'hardware?

Non posso usare il DMA, perché non so in quale momento non lo troverò quel dato nella cache. Il DMA è una cosa che attivo io. Lo uso quando so cosa voglio fare. Se per fare questo usassi un programma sarei lentissimo. Ho messo la cache apposta per andare veloce e poi rallento con il programma. Quindi l'unico modo è usare un meccanismo totalmente hardware. Dal lato del processore si comporta come se fosse una memoria, ma se non ha il dato, dal lato della memoria è come se fosse un processore (o DMA) per prendersi il dato. Prendendolo in memoria farà un protocollo simile al DMA.

Configurazione Pia-memoria



Il sistema di sopra è colui che riceve, il sistema di sotto è colui che trasmette.

La PIA che trasmette manda continuamente i caratteri alla PIA del sistema di sopra che riceve. Poi nel sistema che riceve, i caratteri vengono messi nella MEMORIA-A a partire dall'indirizzo \$00008000

È diverso da fare memoria memoria? Si un po sì. Ha un solo canale ovvero quello che trasmette il messaggio nella memoria, il modo è diverso da specificare. La cosa più difficile da capire è chi setta il modo della PIA. Non è settato dal DMA, ma dal processore. Prima non avevamo questo problema, perché la memoria non va settata.

Il DMA manda l'interruzione appena ha terminato di mettere tutti i caratteri in memoria, quindi ha completato il messaggio.

Quindi nel MAIN devo inizializzare sia la PIA che il DMA

In un sistema con DMA, il filo dell'interruzione della periferica (IRQA) **non è collegato alla CPU**, ma è collegato direttamente al pin di richiesta del DMA (**DREQ0**).

- **Senza DMA:** La PIA alza IRQA. La CPU si ferma, esegue la ISR e Legge il carattere.
- **Con DMA:** La PIA alza IRQA, Il **DMA** vede la richiesta sul suo canale 0, Il DMA chiede il bus alla CPU e Il DMA legge il carattere dalla PIA e lo scrive in RAM.

La CPU rimane nel suo LOOP JMP LOOP (o fa altro) e non sa nemmeno che è arrivato un carattere. L'unico momento in cui la CPU verrà interrotta è alla **fine di tutti i 30 caratteri**, quando scatterà l'interruzione di livello 7 del DMA (quella all'indirizzo \$8700).

Dopo avere settato il controllo del DMA dobbiamo fare questa cosa:

```
MOVE.B    #$08,request(A1)
```

;attiva la richiesta: necessario per il funzionamento con le periferiche simulate da asim (tipo PIA)!

Appena si attiva la richiesta, il DMA comincia a trasferire il contenuto del registro dato associato alla periferica e memorizza in memoria un byte ad ogni colpo di clock. Poichè dal lato trasmittente ci vuole più di una istruzione per scrivere un nuovo dato, lo stesso byte verrà copiato in memoria più volte

Il modo è : 00001000 cioè **single mode** ovvero per ogni byte il DMA deve chiedere il BUS, il trasferimento è **device-memoria**, cioè da un'interfaccia alla memoria. Se invece abbiamo che dalla memoria vogliamo mettere un messaggio nell'interfaccia per mandarlo alla periferica allora è **memoria-device** e il canale 0.

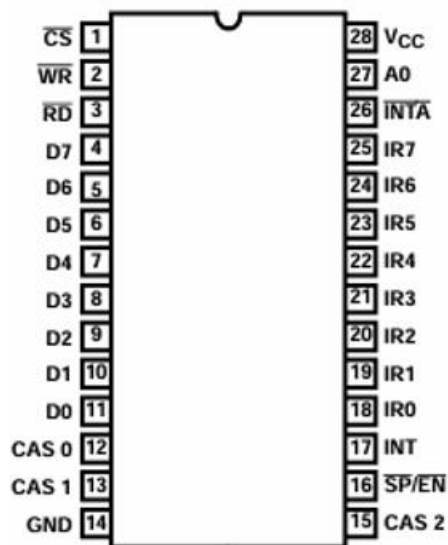
Controllo 10000000 abilitiamo PIA, trasferimento interfaccia-memoria.

Il trasmettitore è un normale sistema con PIA

Qualunque sia il tipo di trasferimento, se il bit 4 di un registro MODE è fissato ad 1, al termine del trasferimento di un blocco i registri CADDR e CCOUNT di quel canale sono caricati con i valori rispettivamente di BADDR e BCOUNT. quindi se il DMA deve gestire il trasferimento di più messaggi, non mi conviene metterlo a 1.

CADDR ha già l'indirizzo successivo all'ultimo scritto, quindi è quello da cui si partirà a scrivere il nuovo messaggio. dobbiamo solo ripristinare CCOUNT che era sceso a 0. Scrivi la lunghezza \$N\$ del nuovo messaggio nel registro CCOUNT. Setta il bit di abilitazione del canale nel registro di controllo del DMA per far ripartire il trasferimento. BADDR non fa nulla che è quello iniziale, tanto quello usato per scrivere è il CADDR

PIC



Il componente Intel 82C59A è un Priority Interrupt Controller (PIC). Può gestire fino a 64 sorgenti di interruzioni vettorzate.

Consente l'implementazione di meccanismi di priorità e mascheramento.

Il pic fornisce un identificativo che serve per poter andare nel vettore e trovare l'ISR. Deve ricevere tante linee di interruzione dai dispositivi.

Il PIC può gestire le priorità rispetto alle linee di interruzione, secondo varie tecniche. Un'altra cosa importante è che non da un'identificativo per ogni periferica che lo interrompe senno dovremmo configurare un sistema mettendo per ogni linea un numero per spostarsi nel vettore delle interruzioni.

Il PIC da una base e rispetto a quella base le linee sono sfalsate.

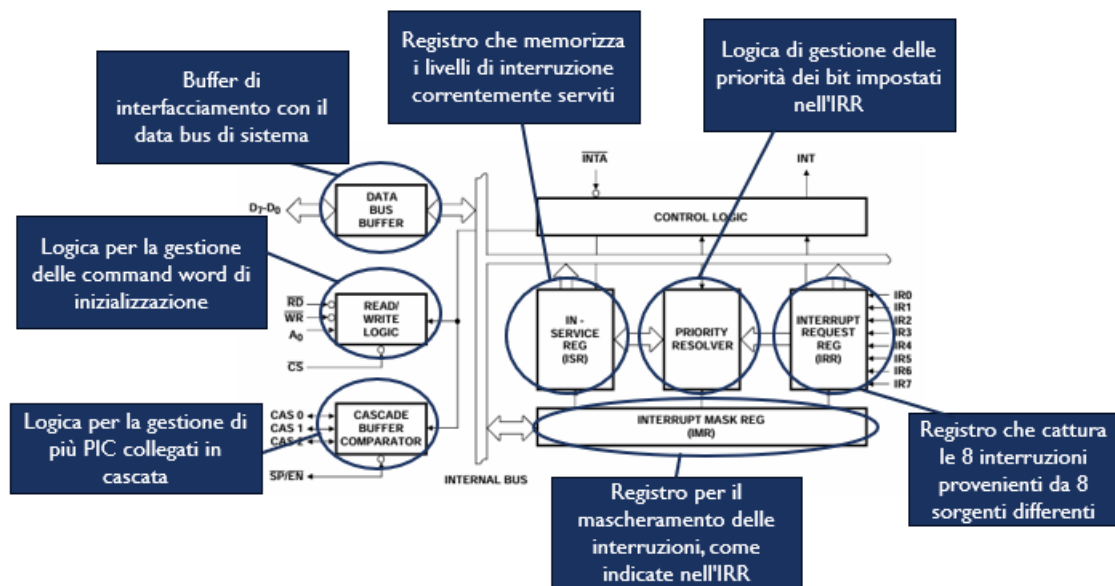
Se la base è 100 la prima linea è a 100, la seconda a 101 etc.

Il PIC ha anche dei registri per mascherare le interruzioni. Devo quindi scegliere la politica di priorità, l'indirizzo base, e se voglio mascherare qualche linea di interruzione.

L'indirizzo base indica il punto da cui mi metterò nella vector table. Le linee sono 7. L'ottava si può utilizzare per gestire dei PIC in cascata. Una delle problematiche del modo è dire al PIC se sta lavorando da solo, ma in cascata perché quando arriva l'interruzione potrebbe non essere stato lui, ma l'altro PIC in cascata.

Nell'architettura notiamo il data BUS Buffer di 8 bit classico che è un buffer bidirezionale che serve per inviare il Vector Address alla CPU e per ricevere i comandi di configurazione.

Cascade Buffer/Comparator: Gestisce i collegamenti "in cascata". Poiché un singolo PIC gestisce solo 8 interruzioni, se ne possono collegare diversi (fino a 9, per un totale di 64 linee). I segnali CAS0-2 servono a identificare quale PIC "slave" deve inviare il proprio vettore quando il PIC "master" riceve un'interruzione.



Il cuore del PIC sono i 3 blocchi che gestiscono le interruzioni:

- **IRR (interrupt request register):** registro delle richieste. Quando una periferica invia una richiesta sulla linea IR0-7, il bit corrispondente viene settato nell'IRR. Indica quali interruzioni sono in attesa di essere servite.
- **Priority resolver:** controlla i bit settati nel IRR e decide quale ha la priorità più alta. Comunica alla control Logic di inviare il segnale INT alla CPU solo se la richiesta è più importante di quella attualmente (eventualmente) in esecuzione.
- **ISR (interrupt service register):** memorizza le interruzioni attualmente in fase di elaborazione. Quando la CPU invia il segnale INTA*, il bit della richiesta prioritaria passa dall'IRR al ISR. Viene azzerato solo quando riceve il segnale di End of interrupt (EOI)

Assumiamo che arrivino tre richieste: una tastiera (IR1), un mouse (IR2) e una scheda di rete (IR3)

IRR vedrà i 3 bit alzati. Il registro avrà questa forma 00001110. Il suo lavoro è finito. **Registra solo le richieste.**

IMR ha il compito di mascherare delle linee. Se mettiamo 00000100 significa che mascheriamo la linea 2. Quindi il priority Resolver vede che nell'IMR quel bit è mascherato, e anche se nel IRR il bit è alto viene ignorato e non arriverà mai alla CPU

Il priority Resolver fa passare IR1 perché più prioritario e alza INT. Il processore l'accetta e risponde con INTA*. Quando arriva questo segnale, il bit 1 nell'IRR si azzerava e si accendeva nel registro ISR 00000010.

Se arrivasse una richiesta del mouse (IR2), il priority resolver la confronterebbe (oltre che con IMR), anche con ISR. Vede che c'è un bit più prioritario alzato e non invia il segnale INT alla CPU. Il bit nel ISR si azzerava in seguito al bit EOI.

Se la richiesta fosse più prioritaria, il priority resolver confronta le richieste 0<1 e lancia interrupt che arriva alla CPU. L'ISR avrà due bit alzati 00000011. La CPU passa alla ISR più prioritaria e quando termina riprende quella precedente.

In ASIM abbiamo una versione semplificata in cui troviamo nell'interfaccia le 8 linee di interruzione e poi ha dei registri.

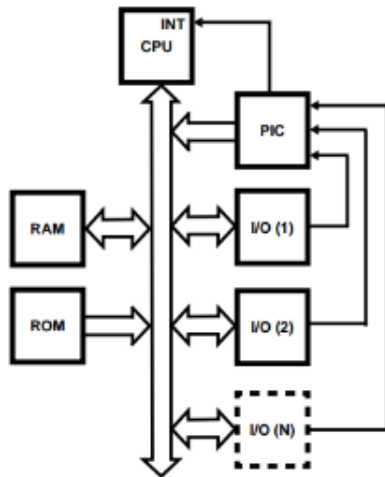


FIGURE 3. INTERRUPT METHOD

Le periferiche (I/O 1, 2 ... N): Sono collegate al bus di sistema principale per lo scambio standard di dati (lettura/scrittura), ma le loro linee di richiesta di interruzione sono separate e cablate direttamente verso il PIC. Queste sono esattamente le linee che entrano nell'IRR (Interrupt Request Register).

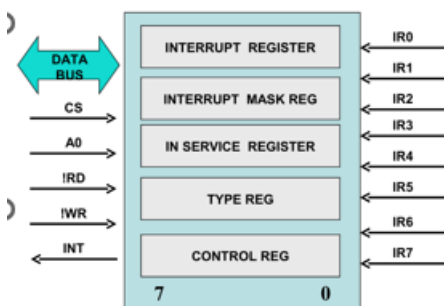
La linea INT (Dal PIC alla CPU): È l'elemento chiave dell'architettura. Non importa quante periferiche I/O ci siano collegate al sistema: il PIC convoglia tutte le richieste in un **unico segnale INT** verso la CPU. In questo modo si evita di dover dotare il microprocessore di innumerevoli pin di interruzione fisici.

Il Bus di Sistema (System Bus): trasporta dati, indirizzi e segnali di controllo. Il PIC vi è collegato perché, dopo aver alzato la linea INT, deve attendere che la CPU gli dia il via libera (tramite un segnale INTA). Una volta ricevuto l'ok, il PIC utilizza il bus dati per inviare alla CPU il **Vettore di Interruzione** (un numero a 8 bit).

PIC SEMPLIFICATO

Il modello di programmazione semplificato del PIC **prevede cinque registri**

- I registri sono associati a indirizzi di memoria (memory-mapped)
 - IRR, IMR, ISR, TR, CTRL**
- L'ordine di accesso (in scrittura) ai registri TR e IMR (indirizzo dispari) avviene sequenzialmente dopo il reset
- L'accesso al CTRL (indirizzo pari) in lettura dipende dai bit 7 (RR) e 6 (RS) del registro di controllo
 - La lettura di IRR e ISR (indirizzo pari) dipende da se RR sia pari a 1 e se RS sia pari a 0 o 1



Abbiamo solo due indirizzi associato Al PIC:

se prendiamo l'indirizzo dispari in scrittura il primo registro dopo il reset è il TYPE REGISTER dove andiamo a scrivere l'indirizzo base da dove far partire i vettori.

Il successivo accesso in lettura/scrittura ad indirizzo dispari è nel IMR.

SE scrivo ad indirizzo pari è sempre il registro di controllo.

Sulla base di cosa scrivo nel registro di controllo, al prossimo accesso in lettura, decido se devo leggere il registro di controllo, il registro IRR oppure ISR.

\$2000 CNTRL/IRR/ISR
\$2001 TR/IMR

Sono tutti registri a 8 bit
IRR E ISR li posso solo leggere
perchè non fanno parte della
gestione del programmatore.

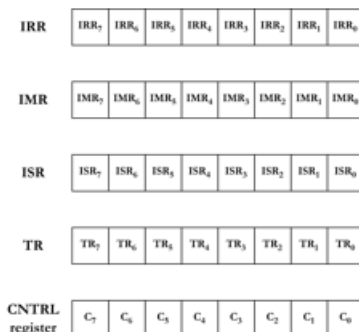
posso ovviamente leggere e scrivere
il controllo

Scrivo il Type Register e posso
leggere/scrivere IMR

Indirizzo	Tipo di accesso	RR ↑ bit 7 del CNTRL	RS ↑ bit 6 del CNTRL	Registro
Pari	W	0/1	0/1	CNTRL
	R	0	0/1	
Pari	R	1	0	IRR
Pari	R	1	1	ISR
Dispari	W	0/1	0/1	TR*
Dispari	W/R	0/1	0/1	IMR

- **IRR**
 - **IRR_{7..0}**->richieste di interruzione
- **ISR**
 - **ISR_{7..0}**->interruzioni trasmesse
- **IMR**
 - **IMR_{7..0}**->interruzioni mascherate

questi 3 registri sono molto semplici perché IRR indica le richieste sottomesse al PIC. Nel registro ISR c'è l'interruzione attualmente servita, mentre nel IMR indica le interruzioni mascherate.



poi dobbiamo vedere il CNTRL register e il TYPE register

CNTRL register

- **C₇**->bit RR
 - **1:** lettura di ISR o IRR
- **C₆**->bit RIS
 - **0:** accesso IRR, **1:** accesso ISR
- **C₅**->non usato
- **C₄**->Automatic End Of Interrupt (AEIO)
 - **1:** cancella automaticamente il bit in ISR dopo la trasmissione dell'interruzione alla CPU
- **C₃**->End Of Interrupt (EOI)
 - **1:** cancella il bit n-esimo puntato da C_{2..0}
- **C_{2..0}**->n-esimo bit da cancellare in ISR

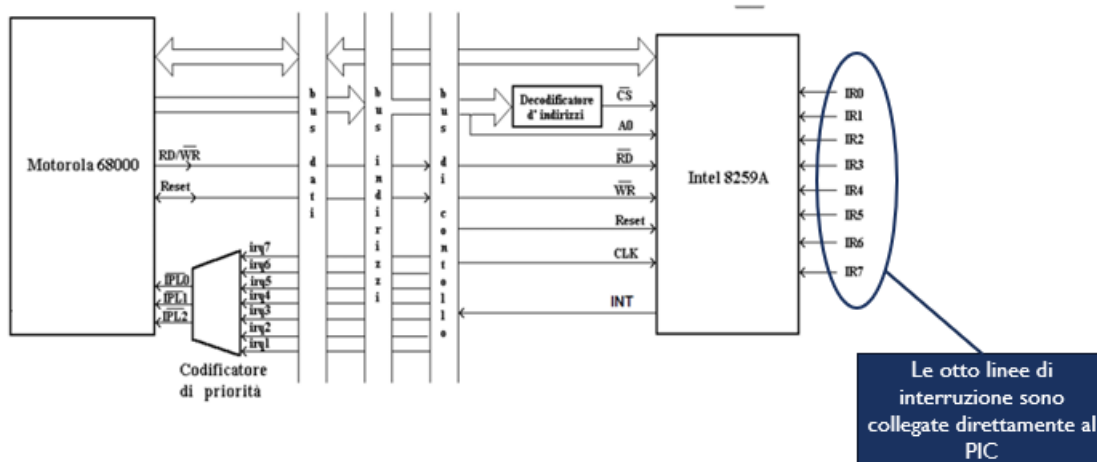
C7 e C6 sono i bit che come abbiamo detto prima determinano i registri che saranno acceduti.

Se in C7 metto 0 allora se l'accesso è all'indirizzo pari è il registro di controllo indipendentemente da C6. Se l'accesso è in scrittura anche C7 è inutile.

Se invece voglio leggere da IRR o ISR metto 1 nel bit C7 e con C6 decido quale registro leggere.

Il registro TR indica semplicemente l'indirizzo base: Per calcolare il primo indirizzo dal quale partiranno le interruzioni utente è sufficiente moltiplicare il valore caricato nel TR per 4. Ad es., se in TR è caricato 128 (10000000), il primo indirizzo utile è 512, ossia \$200.

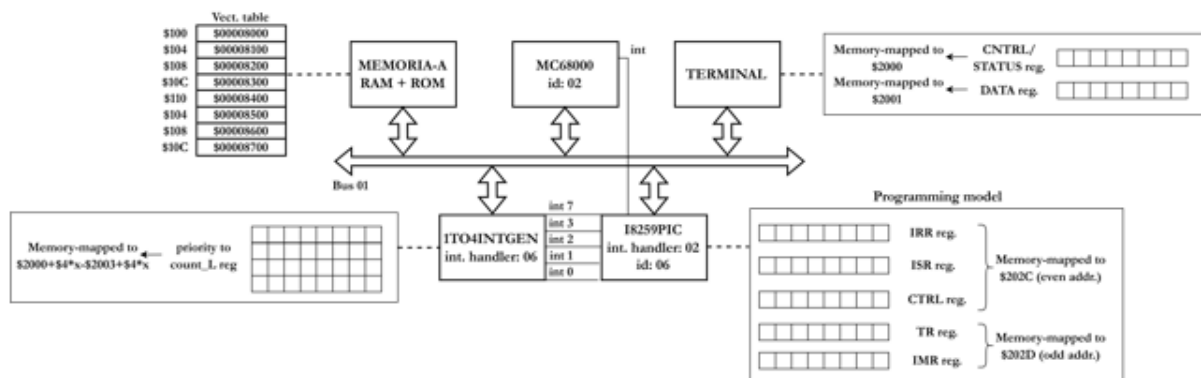
Collegamento con il processore



0001000000000 = \$200 = 512

Quindi in sostanza il PIC ha come gestore delle interruzioni il motorola, mentre i device hanno come gestore delle interruzioni il PIC, esse confluiscono tutte nel PIC.

Configurazione PIC

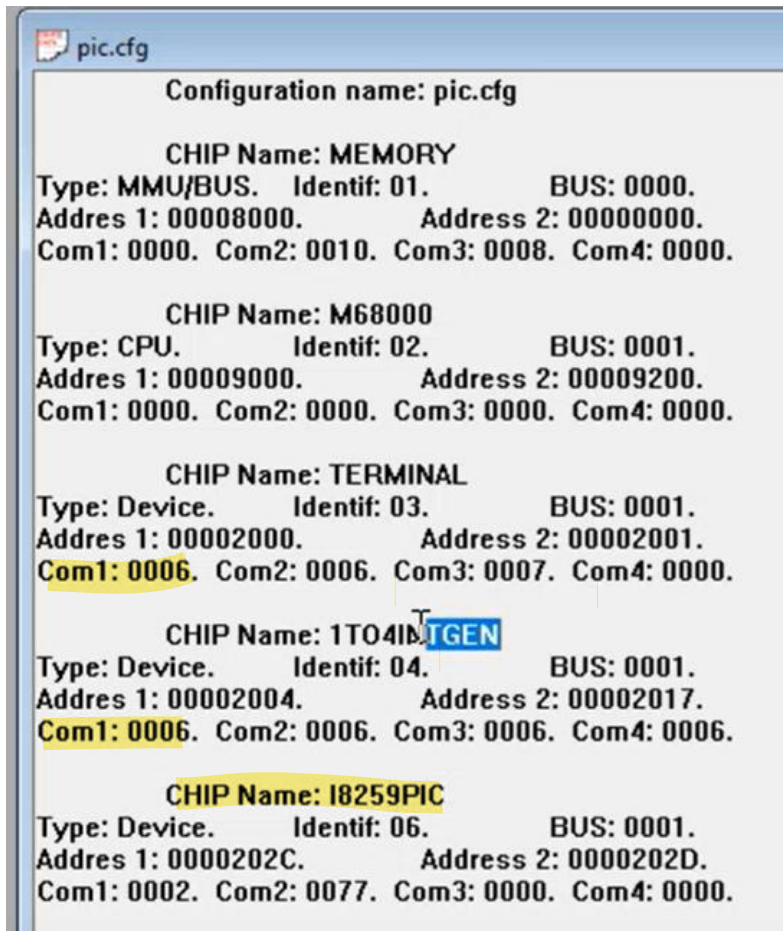


La configurazione istanzia un sistema costituito di tre periferiche: un terminale, un PIC, e un generatore di interruzioni (ITO4INTGEN)

- L'ITO4INTGEN è collegato tramite 5 linee di interruzione al PIC
- La prima interruzione può essere innescata manualmente
- Le altre interruzioni sono generate da dei timer

Nell'architettura vediamo il terminale con i suoi due registri, il PIC con i suoi 5 registri, c'è la memoria in cui si vede la vector table e questo nuovo componente di cui non abbiamo interesse a capire l'architettura, ma è stato creato solo per testare il PIC.

Il generatore ha 5 linee: linea 7 e poi da 3 a 0. La 7 la faremo partire noi, poi ci sono 4 timer che faranno partire le altre interruzioni.



COM1 dice chi è il gestore delle interruzioni. Infatti se vediamo il generatore ha COM4=0006.

Il dispositivo 0006 è l'identificativo del PIC.

Il PIC ha bisogno di due soli registri che vediamo in ADDRESS1 (base), ADDRESS2(ultimo indirizzo). Sono solo due quindi sono sequenziali.

0000202C dispari quindi IMR e TR, mentre pari CNTRL, ISR, IRR.

```
* ***** INIT PIC *****
```

* 1) SCRITTURA NEL REGISTRO TR *****

- * < primo accesso in scrittura all'indirizzo dispari dopo il RESET (i successivi saranno per IMR) >
- * viene settato il valore \$40 = 010001000 che corrisponde al valore 64 (\$40 in esadecimale), che indica
- * il vettore di base (primo vettore utile per le ISR definite da utente). Tale vettore è memorizzato alla locazione $64 \times 4 = 256 = \$100$;

```
MOVE.B    #$40,1(A0)
```

```
* 2) SCRITTURA NEL REGISTRO CTRL *****
```

- * < accesso in scrittura all'indirizzo pari >
- * < i prossimi accessi in lettura saranno al
- *CTRL: se bit7=0, (è questo il caso)
- *IRR: se bit7=1 e bit6=0
- *ISR: se bit7=1 e bit6=1 >

* si pone AEOI=0, quindi appena prima di ritornare dalla ISR è necessario settare manualmente il bit EOI

MOVE.B #\$00,(A0) controllo, indirizzo pari

* 3) SCRITTURA NEL REGISTRO IMR *****

* < scrittura nel registro dispari dopo aver settato TR >

* vengono abilitate tutte le interruzioni

MOVE.B #\$00,1(A0)

* ISR associata alla linea 0

ORG \$8000

int0 MOVE.B #48,(A3) stampa 0 a terminale

MOVE.B #\$08,(A0) setta EOI=1 per resettare il bit 0 di ISR (1000)

RTE

Come scritto prima dobbiamo alzare manualmente EOI perché abbiamo disabilitato AEOI cioè che lo fa in automatico